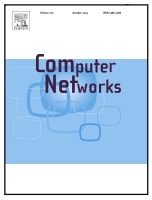
Computer Networks 281 (2026) 112220



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





A simulation-based unified framework for real-time malicious traffic detection in multi-environment networks

Furqan Rustam *, Anca Delia Jurcut *

School of Computer Science, University College Dublin, Belfield, Dublin 04, D06WK27, Dublin, Ireland

ARTICLE INFO

Keywords:
Machine learning
Cybersecurity
Real-time simulation
Client-server communication
Malicious traffic detection

ABSTRACT

A *multi-environment* (M-En) network, integrating various network architectures, faces significant challenges in detecting malicious traffic due to diverse protocols and traffic patterns. Developing separate security frameworks for each network type increases management overhead, limits scalability, and raises costs. Another issue in this domain is the limited deployment-oriented validation of *malicious traffic detection systems* (MTDS), which restricts their effectiveness in protecting M-En networks under live conditions. To address these challenges, this study proposes a unified simulation-based, deployment-feasible, transfer learning-based MTDS for M-En networks, with a focus on both IoT and traditional IP-based infrastructures. In our client-server simulation setup on Ubuntu, the client captures traffic at the central gateway and forwards it to the server for analysis, where flagged malicious packets are discarded at the gateway following batch-level analysis. A representative M-En feasibility dataset is generated using *partial least squares* (PLS) canonical analysis by merging two benchmark datasets, IoT23 (IoT malware traffic) and CICDDoS2019 (traditional IP-based DDoS traffic). An RNN-LSTM-based transfer learning model is employed for feature extraction from this M-En dataset, and various machine learning algorithms are trained and evaluated in both offline and deployment-oriented simulation settings. Logistic Regression emerges as the best-performing model, achieving a mean accuracy of 0.98 in offline testing and 0.71 during simulation-based gateway evaluation, along with the lowest computational time (averaging 0.521 seconds offline and 0.59 seconds per batch during simulated deployment) and minimal memory and CPU usage.

1. Introduction

Multi-environment (M-En) networks, which integrate diverse network architectures under a unified framework, are increasingly being adopted due to their potential to enhance operational efficiency [1]. However, the complexity of protocols across different network types within M-En systems also makes them particularly vulnerable to security threats [2]. These vulnerabilities provide malicious actors with opportunities to compromise network integrity, steal sensitive assets, damage organizational reputations, and disrupt critical services [3]. Traditionally, M-En networks have been protected by segmenting the network and deploying individual *malicious traffic detection systems* (MTDS) for each segment [4]. Although this segmented approach can improve isolation, it significantly increases resource usage, management complexity, and operational overhead [5]. Fig. 1 compares the traditional segmented approach to the unified AI-based approach, highlighting the advantages of integration for M-En network security.

Why M-En Networks and Unified MTDS? Practical Implications. M-En networks are increasingly present in modern infrastructures such as industrial automation systems [6,7] and enterprise environments that integrate offices, warehouses, and server facilities [8]. These environments typically involve multiple network types operating simultaneously:

- **Warehouse environments:** IoT devices (e.g., cameras, automatic locks, smart bulbs).
- **Office environments:** traditional IP-based networks (e.g., computers, printers, and standard networking equipment).

Each of these networks demands distinct security measures. Traditionally, securing them separately results in high management overhead, increased computational and resource costs, and continuous monitoring efforts to ensure security updates [9]. Unified MTDS addresses these challenges by integrating security measures across all networks under a single system. This approach enhances efficiency, reduces costs,

* Corresponding authors.
E-mail addresses: furqan.rustam@ucdconnect.ie (F. Rustam), anca.jurcut@ucd.ie (A.D. Jurcut).

https://doi.org/10.1016/j.comnet.2026.112220
Received 10 March 2025; Received in revised form 7 March 2026; Accepted 12 March 2026
Available online 14 March 2026
1389-1286/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

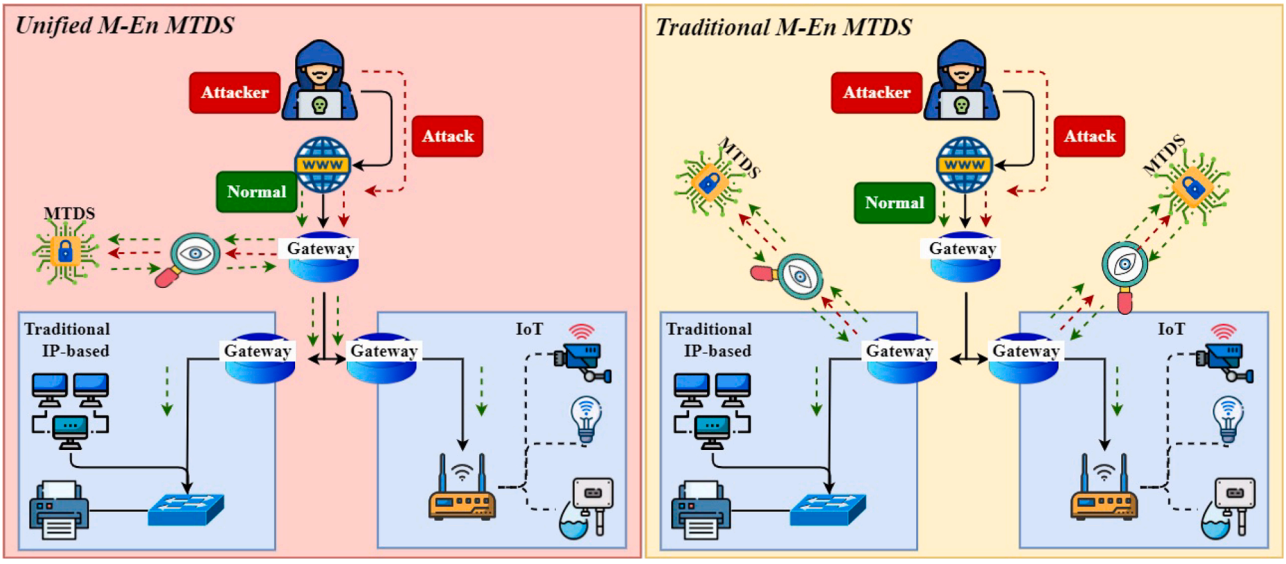


Fig. 1. Traditional vs Unified M-En MTDS.

and simplifies updates and management compared to maintaining multiple independent security solutions. However, challenges remain, such as establishing M-En benchmark datasets, enabling continuous learning, and handling system diversity, which require further research and development.

Recent research in M-En networks is limited, with few studies proposing approaches to manage the diverse traffic within M-En networks; however, these lack real-time testing [1,5,10,11]. For instance, Rustam et al. [1] introduce a *fully automated malicious traffic detection system* (FAMTDS) that utilizes the moth-flame optimizer to enhance machine learning and feature engineering techniques. This study combines IoT and non-IoT (traditional IP-based) datasets, specifically IoTID-20 and UNSW-NB15, to generate a comprehensive M-En dataset. Similarly, another study by Zukaib et al. [10] integrates Federated Learning with Meta-learning in a multi-phase architecture to detect cyberattacks within IoMT networks. Their approach uses advanced clustering and biased classifiers for zero-day attack detection and is deployed on both cloud (IaaS) and fog (SaaS) infrastructures [12], making it adaptable to dynamic and heterogeneous environments. Rustam et al. [5] consider three network types within an M-En network, such as IoT, *traditional* (Tr.) IP-based, SDN, and utilize a self-ensemble model optimized by the particle swarm optimizer to handle traffic diversity within M-En networks.

Although progress is being made in M-En networks, they continue to face unique challenges that require further research to advance the field [13]. One key issue is the lack of benchmark datasets specific to M-En networks. Previous studies often use existing datasets, combining them to simulate M-En scenarios without addressing real-time applications [3,11]. M-En networks encompass various types of networks, such as IoT, non-IoT (Tr. IP-based), SDN, and Industrial IoT, all operating under one framework and involving a diverse range of protocols [14,15]. This diversity in protocols makes it difficult to develop AI-based approaches that can be generalized effectively. For instance, a model trained on IoT traffic may not perform well on other types of network traffic, and vice versa [5]. Therefore, creating an AI-based approach that works well across all network types without compromising the accuracy for any specific traffic type is challenging. Another limitation in the previous studies is the lack of real-time testing and deployment [16]. Most studies have tested their proposed approaches in offline mode using the same dataset for training and testing, which is not ideal for real-time evaluation [17,18]. To the best of our knowledge, limited prior work on the M-En approach has evaluated an MTDS under deployment-oriented conditions using unseen traffic distributions while enabling gateway-level malicious traffic filtering. Ideally, an MTDS should be positioned at the gateway in M-En networks, where traffic can be captured and analyzed using continuous, batch-based decision-making mechanisms to identify and block malicious activity while allowing benign traffic. Addressing these challenges would improve the robustness and practical applicability of AI-based detection systems in M-En networks.

This study advances malicious traffic detection in M-En networks by addressing existing challenges. We propose a simulation-based real-time MTDS that captures traffic from a central gateway, processes it on a server, and discards malicious packets after batch-based analysis at the gateway. Our approach begins with efficient M-En dataset generation using cross-decomposition techniques, where each network's features are combined into a common feature space using *partial least squares* (PLS) canonical analysis [19]. Following this, we apply a *recurrent neural network* (RNN)-*long short-term memory* (LSTM)-based transfer learning approach (RNN-LSTM-TL) to extract sequential patterns and temporal dependencies from the M-En dataset [20]. Several machine learning models are then trained on these transfer-learned features for accurate malicious traffic detection. To enable simulation-based real-time testing, we save the PLS canonical model, RNN-LSTM feature extractor, and trained machine learning classifier as serialized pickle files. A client-server configuration is then established [21], where the server uses these pre-trained models for live simulation-testing of network traffic. The client, positioned at the central gateway, captures network traffic, extracts features from packets, and forwards them to the server to decide on discarding malicious packets. The primary contributions of this study are as follows:

- This study proposes an AI-driven MTDS for M-En networks that collects traffic at a central gateway and processes it using a client-server model. Traffic is logically discarded at the gateway following batch-level analysis when malicious patterns are detected, unlike existing studies that focus solely on offline testing with static datasets [5]. Our approach is trained on a broader range of traffic patterns from IoT and traditional networks (IoT malware and DDoS), improving detection performance on heterogeneous and statistically unseen held-out data.
- We construct a synthetic M-En dataset, addressing the lack of benchmark datasets in M-En research. We created this dataset using IoT23 and CICDDoS2019 datasets, employing PLS canonical analysis to map features into a unified space. This overcomes the limitations

of older datasets [1,5] and ensures our dataset accurately represents M-En traffic, making it a valuable resource for future research.
- Furthermore, we introduce a transfer learning-based approach for feature extraction. While some existing studies have recently used this combination for classification or attack detection [20], we do not employ it for online decision-making due to its higher computational cost. In real-time settings, fast decision-making models are essential. We therefore utilize an RNN-LSTM-based transfer learning method exclusively for feature extraction, which enhances the ability of AI models to learn linearly separable features and improves detection accuracy. Our results demonstrate significant improvements in both offline evaluation and simulation-based deployment settings, highlighting the effectiveness of the proposed approach.
- This study also proposes a real-time client-server simulation framework using Ubuntu to evaluate system performance. The client captures traffic in batches of 400 to 1200 packets, which are analyzed by the AI-based server. Unlike recent studies, our system distributes the workload between client and server, improving efficiency and preventing overload. It detects malicious traffic and applies logical packet discarding following batch-level gateway decisions, while evaluating memory usage, decision-making latency, and processing overhead, critical factors often overlooked in conventional approaches.
- Lastly, we propose and validate a unified MTDS framework for heterogeneous IoT and traditional IP network domains within a centralized gateway architecture. Unlike traditional segmented security deployments that require separate detection pipelines per network type, our framework demonstrates the feasibility of consolidated detection across distinct traffic domains in a single deployment-oriented system, reducing operational overhead while maintaining computational efficiency.

This paper is organized as follows. Section 2 reviews related work on malicious traffic detection in M-En networks and transfer learning, highlighting limitations in real-time evaluation of MTDS. Section 3 describes the proposed MTDS simulation architecture and batch-based decision-making process. Section 4 presents the machine learning pipeline, where models are trained offline and deployed for simulation-based testing. Section 5 discusses the experimental results, comparing offline and simulation-based real-time performance. Finally, Section 6 concludes the paper and outlines future research directions.

2. Related work

In this section, we highlight recent research contributions addressing security challenges in modern networks, with a specific focus on M-En networks and real-time testing of security systems.

2.1. M-En MTDS

Several recent studies focused on optimization to capture the protocol diversity in M-En [1,9–11]. Such as the study [3], which focuses on enhancing security in M-En networks using machine learning approaches. They employed an ensemble learning approach optimized with *particle swarm optimization* (PSO) [22]. The M-En network dataset comprised traffic from IoT, Tr. IP-based and SDN-based networks. Various techniques, including feature selection and data balancing, were used to preprocess the M-En dataset. The proposed PSO-based diverse self-ensemble model achieved a significant accuracy of 0.989. However, their approach to fusing datasets to create the M-En dataset did not focus on the relationships between features from different datasets, nor did it address real-time testing. Similarly, another study [23] addressed security in IoT and Tr. IP-based M-En networks. They utilized two datasets, IoTID20 [24], and UNSWNB15 [25], to generate the M-En dataset. To address dataset imbalance and reduce model overfitting, they introduced a *synthetic data augmentation technique* (S-DATE). Their results showed that the *Random Forest* (RF) model outperformed other machine learning models, achieving an accuracy score of 0.991. In another study [9], AI-generated attacks in M-En networks were addressed by proposing an approach that leverages generative models such as CTGAN [26], CopulaGAN [27], and TVAE [28] to generate synthetic AI-based traffic. The deployed machine learning-based approach aimed to detect both AI-based and human-generated attacks by improving generalization through training on diverse traffic patterns. The study by Rustam et al. [5] proposed a FAMTDS that protects M-En networks with high accuracy and efficiency. They optimized the entire system using a moth-flame optimizer [29], which performed feature engineering and model training based on the nature of the dataset. The proposed approach is specifically designed to handle variability within the dataset, as different protocols offer different features, and FAMTDS can address these variations through neural network-based feature engineering. They also utilized two datasets, IoTID20 [24] and UNSWNB15 [25], to generate the M-En dataset. The approach achieved a significant accuracy of 0.85; however, the study is limited in its validation, particularly lacking in real-time testing. Another recent study [30] proposed a lightweight MTDS for M-En networks named *dual-data trained Light Gradient Boosting Machine* (DDT-LGBM), addressing the data imbalance issue highlighted in prior work [1] by using SVMSMOTE [31]. They first split the dataset into training and testing sets, then trained two LGBM models, one on balanced training data and the other on imbalanced data. The diversity in training sets helped to achieve notable results; however, the study remains limited by the absence of real-time testing and the consideration of recent traffic patterns.

2.2. Real-time simulation approaches

Real-time testing for MTDS is relatively limited in the literature; most studies rely on offline testing, and the few that include real-time testing often show suboptimal performance [18,32,33]. For instance, the study by Murtaj et al. [34] introduces the ReTiNA MTDS for Tr. IP-based networks, which combine CICFlowMeter [35] and RF for scalable cyber-attack detection, trained on the CSECICI-IDS2018 dataset [36]. ReTiNA was tested in a GNS3-simulated LAN environment consisting of a Cisco router [37], switches, and Windows and Kali Linux devices, successfully distinguishing between benign and malicious traffic in a real-world-like setting. The testing phase, conducted in normal traffic conditions on a local network and the University of Camerino network, showed no false positives. However, the real-time performance of the model on malicious traffic remains unverified. Testing with real-time malicious traffic could present challenges for the proposed system, as benign traffic was more prevalent in the dataset for training, and the authors did data augmentation to balance malicious samples. Similarly, Ben et al. [38] introduce INIT, a hybrid simulation environment that combines virtual IoT simulations with real physical IoT devices to enable thorough security testing. INIT can generate realistic IoT data, allowing researchers to monitor network traffic across all layers and observe how different parts of the network behave. Additionally, it allows real IoT devices to be connected to the simulation either over the internet or through a special VLAN [39], making it possible to test real-world attack scenarios. By combining both virtual and physical elements, INIT provides a flexible, realistic platform for assessing IoT security. Although INIT combines virtual and physical IoT modules, the virtual components may not fully replicate the diverse and dynamic behaviors of actual IoT devices and networks. This could lead to discrepancies in testing outcomes when compared to entirely real environments.

The study by Phan et al. [40] presents a *Deep Neural Network* (DNN)-based MTDS specifically designed for SDN environments. Using the NSL-KDD dataset [41], the DNN model was trained on six core features easily captured within SDN's flow-based monitoring structure. For real-time testing, the researchers created a setup where the SDN controller periodically requested flow statistics from OpenFlow switches [42]. This flow data was sent back to the MTDS module for anomaly detection,

where the DNN model analyzed it for intrusion signs. Upon identifying potential threats, the MTDS could actively update the SDN's flow tables to mitigate risks. The model achieved 75.75% accuracy on the NSL-KDD test set [41], though the limited feature set may reduce its ability to detect complex modern traffic patterns. In another study, Jadidbonab et al. [43] developed a real-time testbed for evaluating machine learning-based MTDS within in-vehicle networks. This setup integrates a virtual car simulator using Vector CANoe for CAN bus simulation [44] and CARLA for generating realistic driving data [45], supporting interactions between simulated and physical CAN network components. CARLA sends vehicle data to CANoe, which processes and simulates corresponding in-vehicle responses. The MTDS was tested both on a virtual CAN network with replayed logs and in a live automotive testbed. While the MTDS achieved an accuracy of 76.36% in the virtual environment, real-time testing on the testbed revealed a reduced accuracy of 60.52%, likely due to faster data transmission rates and unaccounted driving behaviors in the real-world test setup. These studies demonstrate that while many MTDS perform well in offline testing, their accuracy tends to decline during real-time testing when exposed to previously unseen data in network traffic. This accuracy reduction may be attributed to variations in traffic patterns across different intervals and the potential inadequacy of training data during offline model preparation.

2.3. Transfer learning for MTDS

The use of transfer learning is gaining prominence in network security [46–48]. For instance, Latif et al. [49] proposed a transfer learning approach for industrial IoT networks. They employed a CNN model with a *genetic algorithm* (GA) [50] and bootstrap aggregation ensemble technique on the Edge_IIoTset dataset [51], which they converted into images for experimentation. Hyperparameters of the machine learning models were optimized using GA, resulting in an accuracy score of 1.0. However, converting the traffic into images beforehand could incur higher computational costs, and real-time deployment presents an additional associated challenge in the proposed approach. Similarly, Kim et al. [52] introduced a transfer learning approach for IoT network security. They utilized a 1D-CNN to extract features correlated with target classes, achieving an accuracy score of 0.9994 on the BoT-IoT dataset [53]. Another study by Hoang et al. [54] focused on a transfer learning approach for enhancing in-vehicle network security. They employed a contrastive learning framework where an autoencoder was utilized by the projector to transform data into another feature space. Subsequently, a classifier was trained on the feature space generated by the encoder. Another study [55] proposes a h*ybrid intrusion detection model* (HIDM) using an optimized CNN-LSTM with transfer learning, achieving high precision in multi-class classification on industrial datasets ToN-IoT [56] and UNSW-NB15 [25]. The model operates in hybrid *industrial IoT* (IIoT) environments, where transfer learning helps adapt to the diversity of IIoT settings. In this approach, CNN is used for feature extraction, followed by LSTM for attack prediction. These studies demonstrate the significance of transfer learning in malicious traffic detection. However, most of them applied it to individual network datasets. Additionally, many existing studies employ either LSTM or CNN models for feature extraction [57]. In contrast, our approach employs an RNN-LSTM ensemble that effectively captures complex temporal dependencies and long-range patterns in sequential data, which makes it particularly well-suited for time-series and network traffic analysis.

2.4. Role of feature selection in improving MTDS

Effective feature engineering can significantly enhance the accuracy of malicious traffic detection models [58]. By selecting and transforming relevant features, models can more effectively distinguish between benign and malicious network traffic [59]. Feature selection methods, including extra tree-based selection [5], Gini index-based selection [60], and feature reduction techniques like PCA [11], SVD, and *chi-square* (chi2), are commonly used by researchers to preprocess and reshape datasets. For example, study [11] applied PCA to simultaneously reduce the feature space of the IoTID20 [24] and UNSWNB15 [25] datasets, allowing them to combine these datasets to create an M-En dataset. They then used a stacked ensemble algorithm for attack classification, achieving an accuracy score of 0.985 for the M-En network. Similarly, another study [61] proposed an LSTM-based MTDS that used both linear and nonlinear feature reduction techniques, such as PCA and autoencoders, to generate a fused feature set, which improved machine learning model performance. This approach, tested on the NSL-KDD [41], UNSWNB15 [25], CICIDS2017 [62], and MSCAD [63] datasets, demonstrated enhanced performance over previous studies. Recent research demonstrates that by employing advanced feature engineering techniques, MTDS can significantly enhance their ability to accurately and efficiently identify threats in real-world network environments, even when dealing with limited computational resources or the demands of real-time detection [59,64]. Table 1 shows the summary of related work.

2.5. Limitations & gaps:

Several recent studies have applied state-of-the-art approaches in M-En networks and specific network types. However, these studies often use outdated datasets, such as IoTID20 [24] and UNSWNB15 [25], which may not accurately represent recent attacks. In contrast, our study focuses on recent network threats and utilizes up-to-date datasets, such as IoT23 [66] and CICDDoS2019 [67], to construct the M-En dataset. Another limitation in recent research is the method used to generate M-En datasets from benchmark datasets. For example, [11] applies PCA separately to IoT and Tr. IP-based network datasets without considering the correlations between them, while [3] only performs feature selection before combining datasets, neglecting these correlations. Our study addresses this gap by emphasizing the importance of a shared, correlated feature space between IoT and Tr. IP-based network datasets effectively generate M-En datasets using cross-decomposition methods. Previous studies have also overlooked transfer learning-based feature extraction in M-En networks, which could improve performance since transfer learning has proven effective in malicious traffic detection. Furthermore, a major limitation in most studies is the lack of validation in real-time scenarios. Many cybersecurity studies perform only offline testing, without assessing how their approaches perform in real networks [18,32], their accuracy in real-time settings, the prediction time, or the resources (e.g., memory) required in real-time simulation frameworks. Our study addresses these limitations by proposing a deployment-oriented simulation-based MTDS for M-En networks with high accuracy and efficiency.

3. Proposed real-time simulation framework

This research proposes a simulation-based MTDS for M-En networks, addressing the need to secure diverse network architectures under a unified approach. The system operates in a client-server configuration, where traffic is collected at a central gateway by the client and forwarded to the server for machine learning-based analysis. Malicious traffic is handled through logical packet discarding following batch-level decision-making, as illustrated in Fig. 2. The proposed framework is implemented on Ubuntu using a client-server architecture [68].

3.1. Client-side framework

The client, positioned at the M-En network's central gateway, plays a critical role in capturing and transmitting network traffic for analysis. Although we use pre-extracted features from benchmark datasets (IoT23 [66] and CICDDoS2019 [67]), the client is designed to mimic real-world conditions where packet capture, pre-processing, and packet parsing would occur in real-time. This architecture allows for easy deployment of future PCAP data without major modifications. The client

Table 1
Summary of related work.

Study	Dataset	Method	FE	TL	RTT	M-En	Comparison
[1]	IoTID20, UNSWNB15, InSDN	PSO-based D-SEM	Feature selection (ETC), data balancing (S-DATE)	×	×	✓	Limited to offline testing; resource-intensive, lacks transfer learning, and relies on outdated datasets.
[5]	IoTID20, UNSWNB15	FAMTDS with MFO	NN, ETC	×	×	✓	Lacks real-time validation, relies on outdated datasets, generates M-En dataset without considering feature correlations, and uses a highly complex approach.
[23]	IoTID20, UNSWNB15	S-DATE	DA	×	×	✓	Relies on outdated datasets, lacks feature fusion for M-En data generation, does not incorporate transfer learning, and has limited generalization.
[30]	IoTID20, UNSWNB15, InSDN	DDT-LGBM	SVM-SMOTE	×	×	✓	Limited to offline testing, data balancing can cause for data leakage. lacks transfer learning, and relies on outdated datasets.
[49]	Edge_IIoTset	CNN with GA	Image-based FE	✓	×	×	Computationally intensive due to image conversion, limited to a specific network, and lacks real-time testing.
[52]	BoT-IoT	1D-CNN for FEx	FEx only	✓	×	×	Restricted to CNN for feature extraction; lacks the RNN-LSTM-TL sequential processing in our approach for improved temporal analysis, and does not support real-time testing.
[54]	Custom In-Vehicle Dataset	CL with AE	Contrastive feature transformation	✓	×	×	Focuses on in-vehicle data, lacks real-time validation, and is limited to a specific network.
[55]	ToN-IoT, UNSWNB15	Hybrid CNN-LSTM with TL	Fused FEx using CNN-LSTM	✓	×	×	Lacks real-time validation, focuses on individual datasets, and relies on outdated data.
[34]	CSECICI-IDS2018	CICFlowMeter with RF	FS via RF	×	✓	×	Tested in GNS3 with Cisco router, switches, Windows & Kali devices; effective but lacks testing across diverse network types such as M-En networks
[38]	Virtual and Real IoT Data	INIT - Hybrid Simulation with Real IoT	Traffic monitoring across layers, VLAN/Internet	×	✓	×	Supports practical attack scenarios in IoT, robust for IoT security but limited to IoT-specific environments.
[65]	Mininet (Packet Captures)	Mininet Emulator with Tcpreplay	Synthetic traffic generation via CTGAN	×	✓	×	Real-time limits applicability to broader networks. Limited to specific networks, and used synthetic data also which may not represent real work traffic.
[40]	NSL-KDD	DNN for SDN MTDS	Flow-based monitoring, SDN flow control	×	✓	×	Limited feature set and SDN-specific approach may miss complex modern traffic patterns. They didn't consider M-En networks.
[43]	Real-time In-Vehicle Data	In-vehicle Testbed with CAN Simulation	Vector CANoe, CARLA driving data integration	×	✓	×	Good accuracy in virtual setup; but low in real-time testing, possibly due to faster transmission and unmodeled behaviors in real-world data.
Proposed MTDS	IoT23, CICDDoS2019	RNN-LSTM-TL with ML	Cross-decomposition feature fusion	✓	✓	✓	Incorporates real-time validation, sequential and spatial processing, and transfer learning for feature extraction; utilizes the latest datasets, supports M-En networks, and emphasizes resource efficiency.

FE—Feature Engineering; **TL**—Transfer Learning; **RTT**—Real-Time Testing; —- **Acronyms**: **D-SEM**—diverse self-ensemble model; **MFO**—Moth-Flame Optimization; **NN**—Neural Network; **DA**—Data Augmentation; **GA**—Genetic Algorithm; **FEx**—Feature Extraction; **AE**—Autoencoder; **CL**—Contrastive Learning; **CL**—Transfer Learning; **FS**—Feature selection; —- **Symbols**: ✓ present, × lacking.

captures traffic in randomly sized batches (e.g., 400 to 1200 packets) to simulate real-time variability. In actual deployment, this process would involve extracting packet-level features such as packet length, header flags, and inter-arrival times. Here, however, pre-extracted features simulate this step to expedite testing. After preparing each batch, the client serializes the data using Python's pickle module for efficient transmission [69]. A TCP socket connection is then established on port 8081, using Python's socket library to provide reliable, ordered delivery of packets, an essential requirement to maintain data integrity and batch structure during transmission [70,71]. This setup ensures minimal packet loss, enabling accurate downstream processing. After sending the traffic to the server, the client will wait for predictions from the server.

Once predictions are received from the server, the client acts on the classification. Traffic flagged as malicious is discarded, while benign packets are allowed to flow through the network. This immediate decision-making minimizes potential damage from malicious traffic, maintaining network integrity without significant delay.

3.2. Server-side framework

The server is designed to handle complex processing tasks, leveraging multiple machine learning techniques optimized for malicious traffic detection. On the server, a dedicated socket listener on port 8081 accepts incoming connections, receiving serialized data batches from the client. Upon receipt, the server deserializes each batch [69], reconstructing the data into a format suitable for feature transformation and analysis. This efficient deserialization pipeline ensures that incoming data is quickly prepared for real-time processing [72]. A sequence of pre-trained models is then applied to the data. First, PLS canonical aligns and transforms features from different network types into a unified feature space. Next, the RNN-LSTM-TL model extracts complex, sequential

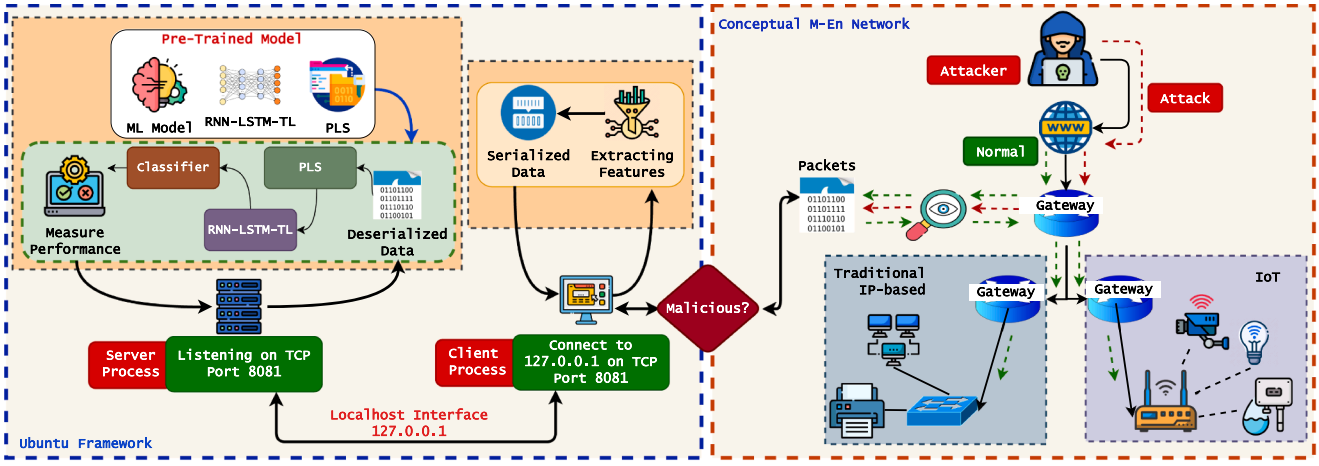


Fig. 2. Proposed methodology diagram for real-time simulations.

patterns from the unified feature set, enriching the features for further analysis. Finally, the refined features are passed to a machine learning classifier, which detects malicious traffic based on these extracted patterns. The server will send predictions on incoming traffic back to the client, enabling it to make decisions about discarding malicious traffic.

3.3. Client-server data flow and real-time decision-making

The client-server interaction creates a continuous feedback loop that detects malicious traffic. Each batch of predictions received by the client informs real-time actions, where malicious packets are discarded at the gateway while benign traffic is allowed to pass. By centralizing model inference on the server, this design effectively balances computational load and minimizes latency, allowing the client to focus on traffic capture and forwarding. The client-server setup also optimizes resource use by assigning real-time capture and transmission tasks to the client and leaving computation-intensive classification to the server. Our client-server architecture supports scalability, as server-side model updates can be applied without reconfiguring the client. It can also be easily migrated to the cloud or modified as needed, making it adaptable to evolving threats in real-time environments. Fig. 3 shows the flowchart diagram for client-server architecture and its process flow inside it.

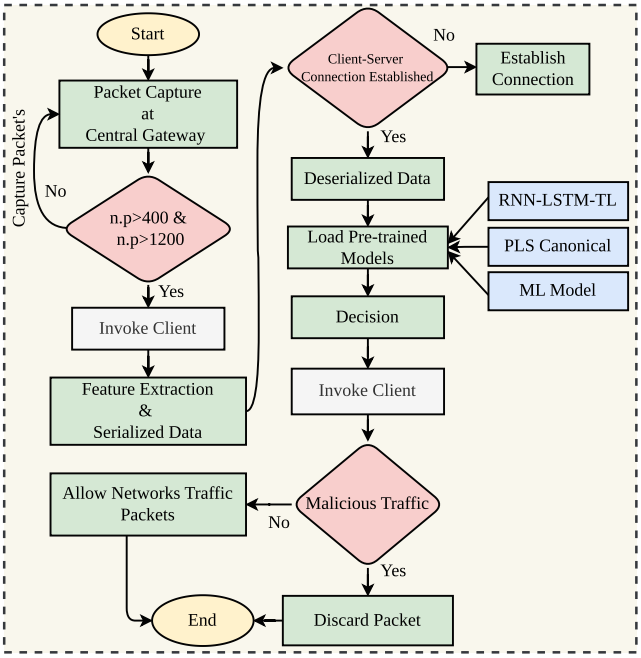


Fig. 3. Flowchart diagram for client-server architecture.

3.4. End-to-end latency decomposition

In the proposed framework, real-time operation is realized through continuous, batch-based processing of network traffic at the gateway, resulting in an end-to-end latency determined by multiple sequential processing stages. Network traffic is buffered into short windows (typically 400-1200 packets), and malicious traffic is handled through logical packet discarding following batch-level analysis. End-to-end latency is evaluated at batch granularity and decomposed into the following stages: (i) batch-level traffic capture and buffering at the client, (ii) feature preparation and serialization, (iii) data transfer between client and server, (iv) feature transformation and model inference at the server, (v) decision feedback, and (vi) logical enforcement through batch-level packet discarding at the application layer. Client-server communication is implemented over a localhost interface to isolate computational overhead; consequently, networking effects such as RTT variation, packet loss, and background traffic contention are excluded. Accordingly, the reported latency values represent lower-bound estimates of batch-level end-to-end processing delay under simulated gateway conditions, rather than packet-level or line-rate intrusion prevention performance.

4. Machine learning pipeline

Our proposed machine learning-based approach for malicious traffic detection in the M-En network using transfer learning methods is shown in Fig. 4. In this approach, we first generate the M-En dataset using IoT and Tr. IP-based datasets. We combine them using feature selection and feature decomposition. Important features are extracted using the *extra trees classifier* (ETC) to select significant features. Then, we extract correlated feature spaces using the PLS cross-decomposition method. This M-En dataset represents two targets: Attack and Benign, which represent both IoT and Tr. IP-based network traffic (see Section 4.1.1). After generating the dataset, we employ a transfer learning approach in which we train RNN-LSTM-TL on the M-En dataset. After training the RNN-LSTM-TL, we use it to extract feature patterns from the PLS generated M-En data by freezing the last layer of the RNN-LSTM-TL (see Section 4.2). After extracting features, we pass this feature set to train several machine learning models. For this, we split the dataset into training and testing sets with an 80:20 ratio. We utilize 80% of the data for model training and the remaining 20% for evaluating the performance of the

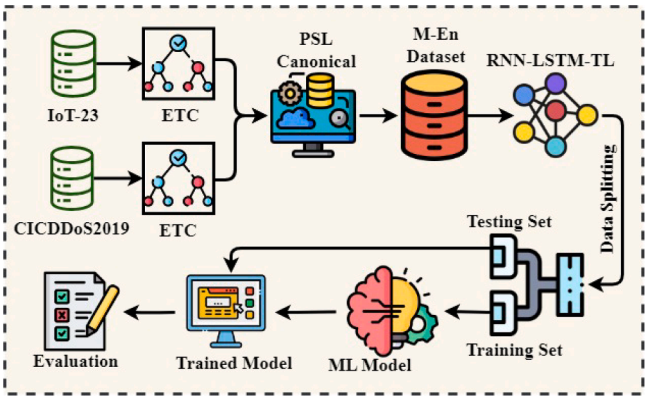


Fig. 4. Proposed methodology for machine learning approach.

learning models. We evaluate the performance of the models in terms of different evaluation metrics such as recall, precision, F1 score, and accuracy, as well as computational time.

4.1. M-En dataset

In this study, we generated a more recent and comprehensive M-En dataset, distinguishing it from previous studies. Unlike earlier approaches, we incorporated recent datasets and applied feature correlation analysis to enhance datasets fusion, resulting in a more efficient M-En dataset. We used a cross-decomposition method and combined two benchmark datasets, as described below:

4.1.1. IoT23 & CICDDoS2019

In this study, we used two recent datasets representing different network environments. The IoT23 dataset [66] contains packet captures of IoT devices infected with various malware families such as Mirai, Gafgyt, Torii, and Okiru. It includes 1,446,621 samples, of which 53,481 are labeled as 'Benign' and 1,393,140 as 'Attack', reflecting behavior typical of compromised IoT devices rather than general-purpose networks. The CICDDoS2019 dataset [67] contains 431,371 samples, with 333,540 labeled as 'Attack' and 97,831 as 'Benign'. This dataset focuses specifically on structured DDoS attacks, including reflection-based and exploitation-based TCP/UDP flood vectors, captured in traditional IP networks. For the experimental evaluation, we selected 50,000 samples from each dataset to maintain balanced computational requirements. As both datasets primarily cover IoT malware and DDoS traffic, the combined M-En dataset serves as a feasibility study rather than a fully generalizable representation of all network attack types.

4.1.2. Methodology for M-En dataset generation

To generate the M-En dataset, we utilized the cross-decomposition method, specifically the PLS canonical [73,74]. PLS canonical captures correlations between variables in both datasets, resulting in a more comprehensive merge, improving subsequent analyses and model performance [73]. We chose PLS canonical for its ability to work with both categorical and numerical variables. Before deploying PLS, we selected important features using the ETC, which helps in selecting features that are more correlated and contribute to predicting the target accurately [3]. We applied a filter on the importance score with a threshold of 0.001. Any feature with an importance score greater than 0.001 was selected for further experimentation. From the CICDDoS2019 dataset, ETC selected 43 features out of 80, and from the IoT-23 dataset, selected 19 features out of 23. Following this, we deployed the PLS canonical approach to cross-decompose the datasets into the M-En dataset.

The PLS canonical method involves several steps to generate the M-En dataset. Both datasets X_{IoT} and $Y_{tr.IP}$ are first normalized to have zero mean and unit variance to ensure comparable feature scales. The

Table 2
Dataset samples count.

Dataset	Total	Benign	Attack
IoT23	50,000	23,215	26,785
CICDDoS2019	50,000	7253	42,747
M-En	100,000	30,468	69,532

PLS canonical correlation analysis then projects both datasets into a shared latent space by maximizing the covariance between their corresponding components. Let W_x and W_y denote the weight matrices for X_{IoT} and $Y_{tr.IP}$, respectively. The latent feature representations are obtained as:

$$T = X_{IoT} W_x, \quad U = Y_{tr.IP} W_y \tag{1}$$

The PLS canonical algorithm iteratively updates W_x and W_y to maximize the covariance between T and U:

$$\max_{W_x, W_y} \operatorname{cov}(T, U) = \operatorname{cov}(X_{IoT} W_x, Y_{tr.IP} W_y) \tag{2}$$

The resulting canonical features T and U capture the most correlated structures between IoT and Tr. IP-based traffic. These components are then combined to form the unified M-En feature representation, which serves as input for downstream training and evaluation. Table 2 present sample counts per class in each dataset.

For reproducibility, we have made available a supplementary script in our GitHub repository[1], which implements the M-En data generation process described above, to ensure that other researchers can replicate the dataset generation procedure and extend our experiments.

4.2. RNN-LSTM-TL features

The RNN-LSTM-TL model is designed to extract temporal feature representations for training machine learning models. This ensemble architecture effectively captures sequential dependencies in network traffic data, making it well-suited for detecting patterns indicative of malicious activity. Although CNNs are widely employed in malicious traffic detection for their ability to extract local patterns [75,76], they are limited in modeling temporal dependencies across longer sequences [77]. In contrast, our combined RNN-LSTM architecture is explicitly designed to capture both short-term and long-term sequential relationships in network traffic. The RNN component models the recurrent dependencies across packet flows, while the LSTM units retain longer-term context that is critical for identifying evolving attack behaviors. Specifically, the LSTM component within RNN-LSTM-TL enables the model to learn long-term dependencies, allowing it to detect subtle attack patterns that span multiple network packets or sessions [78]. This combination makes RNN-LSTM more suitable than CNN-based approaches for malicious traffic detection in M-En networks, where temporal correlations are a defining characteristic of attack patterns. The RNN-LSTM-TL approach simply takes network packet features as input. In our case, we have 15 M-En data features. Let the input sequence be represented as:

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}, \quad \mathbf{x}_t \in \mathbb{R}^{15} \tag{3}$$

where X is the input sequence with T time steps, and each feature vector $\mathbf{x}_t$ has 15 input features, corresponding to the network traffic attributes. The RNN-LSTM-TL architecture consists of five layers. The first layer is an RNN with 128 units, which processes sequential data while maintaining a hidden state over time. The hidden state at each time step t is computed as:

$$\mathbf{h}_t^{(RNN)} = \tanh(W_h^{(RNN)} \mathbf{h}_{t-1}^{(RNN)} + W_x^{(RNN)} \mathbf{x}_t + \mathbf{b}^{(RNN)}) \tag{4}$$

[1] https://github.com/furqanrustam/M-En-PLS-and-RNN-LSTM-Ubuntu-Simulation

where $W_h^{(RNN)} \in \mathbb{R}^{128\times128}$ and $W_x^{(RNN)} \in \mathbb{R}^{128\times15}$ are the weight matrices, and $\mathbf{b}^{(RNN)} \in \mathbb{R}^{128}$ is the bias term. The hidden state $\mathbf{h}_t^{(RNN)}$ has 128 units. Following the RNN layer, we introduce an LSTM layer with 64 units, which captures long-term dependencies. The LSTM cell consists of three gates: input gate i_t, forget gate f_t, and output gate o_t, which regulate the memory cell C_t:

$$f_t = \sigma(W_f^{(LSTM)}\mathbf{x}_t + U_f^{(LSTM)}\mathbf{h}_{t-1}^{(LSTM)} + b_f^{(LSTM)})$$

$$i_t = \sigma(W_i^{(LSTM)}\mathbf{x}_t + U_i^{(LSTM)}\mathbf{h}_{t-1}^{(LSTM)} + b_i^{(LSTM)})$$

$$o_t = \sigma(W_o^{(LSTM)}\mathbf{x}_t + U_o^{(LSTM)}\mathbf{h}_{t-1}^{(LSTM)} + b_o^{(LSTM)})$$

$$\tilde{C}_t = \tanh(W_C^{(LSTM)}\mathbf{x}_t + U_C^{(LSTM)}\mathbf{h}_{t-1}^{(LSTM)} + b_C^{(LSTM)})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$\mathbf{h}_t^{(LSTM)} = o_t \odot \tanh(C_t)$$

where, weight matrices are $W_i^{(LSTM)}$, $W_f^{(LSTM)}$, $W_o^{(LSTM)}$, $W_C^{(LSTM)} \in \mathbb{R}^{64\times128}$, and $U_i^{(LSTM)}, U_f^{(LSTM)}, U_o^{(LSTM)}, U_C^{(LSTM)} \in \mathbb{R}^{64\times64}$. Bias terms are $b_i^{(LSTM)}$, $b_f^{(LSTM)}$, $b_o^{(LSTM)}$, $b_C^{(LSTM)} \in \mathbb{R}^{64}$. To prevent overfitting, we apply a dropout layer with a dropout rate of 0.5, which randomly deactivates 50% of neurons during training [79]. The dropout operation is defined as:

$$\mathbf{h}_t^{(Dropout)} = M \odot \mathbf{h}_t^{(LSTM)} \tag{5}$$

where $M \sim \text{Bernoulli}(p = 0.5)$ is a dropout mask sampled from a Bernoulli distribution. Next, the dense layer with 32 neurons applies a ReLU activation function, defined as:

$$\mathbf{h}^{(Dense)} = \max(0, W_d\mathbf{h}_t^{(Dropout)} + b_d) \tag{6}$$

where $W_d \in \mathbb{R}^{32\times64}$ and $b_d \in \mathbb{R}^{32}$. The final layer is a single-neuron dense layer with a sigmoid activation function, which outputs a probability score for binary classification:

$$\hat{y} = \sigma(W_o\mathbf{h}^{(Dense)} + b_o) \tag{7}$$

where $W_o \in \mathbb{R}^{1\times32}$ and $b_o \in \mathbb{R}$. The sigmoid function is given by:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{8}$$

which ensures the output probability lies between 0 and 1 for distinguishing between Benign and Attack classes. The model is trained using binary cross-entropy loss, which is formulated as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{9}$$

where N is the number of training samples, y_i is the true label, and $\hat{y}_i$ is the predicted probability. The Adam optimizer is used to update model parameters, with a learning rate of 0.001, ensuring efficient gradient updates [80]. The model is trained for 100 epochs with a batch size of 32. After training, we freeze the last layer and extract the output from the second-to-last dense layer, yielding a 32-dimensional feature representation for each sample:

$$\mathbf{F} = \{f_1, f_2, \ldots, f_{32}\}, \quad f_j \in \mathbb{R} \tag{10}$$

where $\mathbf{F}$ represents the extracted feature vector. These deep features are then used as input to traditional machine learning models for final classification. The architecture of the RNN-LSTM-TL model is illustrated in Fig. 5. The RNN-LSTM-based transfer learning module captures both short- and long-term temporal dependencies in network traffic, enabling rich, time-aware feature representations for sequential pattern recognition. These learned features improve the detection of evolving malicious behaviors by enhancing linear separability across heterogeneous IoT and traditional IP traffic sources in the merged M-En dataset. The RNN-LSTM module is used solely for representation learning during inference and is not responsible for decision-making; instead, lightweight classifiers are applied to the extracted features to perform efficient batch-level gateway decisions during deployment.

Fig. 6 shows the data representation after each step in feature engineering. Initially, we have the original feature set and values, with the IoT23 and CICDDoS2019 datasets containing 19 and 43 features, respectively, after feature selection using ETC. After applying PLS, the data is converted into a unified M-En dataset with a fused feature set consisting of 15 features. Finally, RNN-LSTM-TL transforms these features into valuable representations, resulting in a final dataset with 32 features after transfer learning.

Table 3
Machine learning models' hyperparameters.

Model	Hyperparameters	Tunning

RF	`n_estimators=300`
	`random_state=2`
	`max_depth=50`
		`n_estimators= [20 to 500]`
		`random_state= [1 to 20]`
		`max_depth=[2 to 100]`

LR	`solver='liblinear'`
	`multi_class='ovr'`
	`C=2.0`
		`solver=['liblinear' & 'saga' ]`
		`solver='ovr'`
		`[C=1.0 to 5.0]`

KNN	`n_neighbors=7`	`n_neighbors=[3 to 15]`
LGBM	`random_state=2` `n_estimators=300`	`random_state=[1 to 20]` `n_estimators=[20 to 500]`
GNB	Default	–

4.3. Machine learning algorithms

We utilized five machine learning models following feature extraction using RNN-LSTM-TL, including RF [81], LR [82], KNN [83], LGBM [1], and GNB [84]. These methods were applied with different hyperparameter settings, as outlined in Table 3. We determine the optimal hyperparameter settings using grid search methods and literature insights, train models within a specific range for optimal performance [85]. For tree-based models RF and LGBM, the *n_estimators* parameter, set to 300, facilitates a robust ensemble of decision trees [23], with a random state of 2 ensuring reproducibility. Additionally, a *max_depth* of 50 was specified to prevent overfitting by limiting the depth of each tree. In LR, the 'liblinear' solver was chosen for its efficiency in handling binary classification tasks, while KNN utilized *n_neighbors* set to 7 to balance model complexity and the bias-variance trade-off. Finally, GNB is employed with a default hyperparameter setting.

4.4. Training and evaluation protocol

To prevent data leakage, all feature alignment and representation learning steps are trained exclusively on the training data. The PLS canonical model is fitted only on the training split and subsequently saved. During offline testing and simulation-based real-time evaluation, the learned PLS transformation is applied using the fixed transformer without refitting. Similarly, the RNN-LSTM-based transfer learning feature extractor is trained solely on the training data and saved after convergence. This model remains frozen during both offline testing and real-time evaluation, where it is used only for feature extraction without any further parameter updates. Hyperparameter tuning for all models is conducted using the training data only, and the selected hyperparameters remain unchanged throughout offline testing and real-time evaluation. The same pre-trained models are consistently used for all evaluation scenarios, ensuring that no information from test or real-time data influences model training or parameter selection.

All offline model training and hyperparameter tuning were performed on a Windows-based host machine equipped with a 12th-generation Intel Core i7 processor, 64 GB of RAM, and a 1 TB SSD. Simulation-based deployment and performance evaluation were conducted on an Ubuntu 22.04.5 LTS operating system running inside a

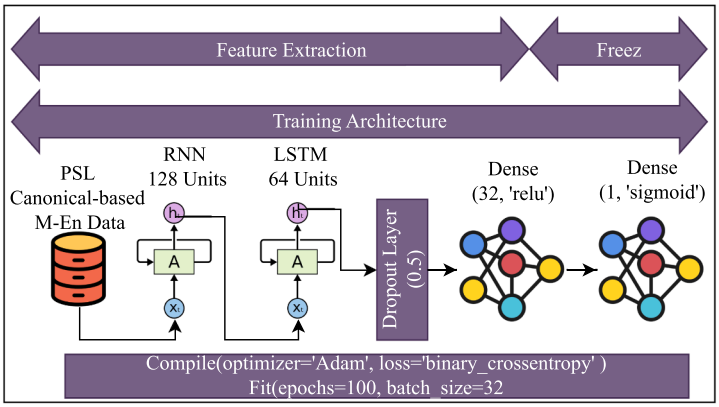
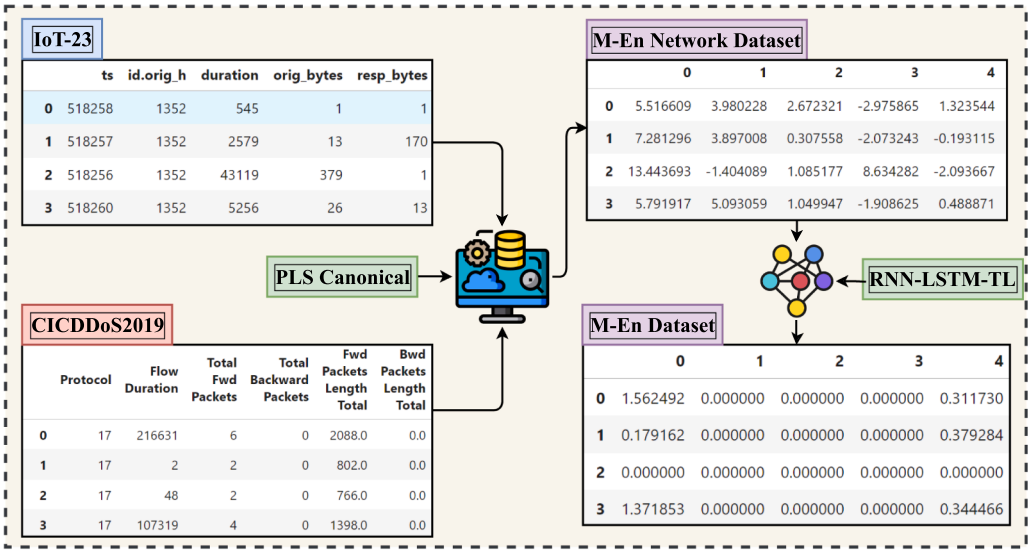


Fig. 5. RNN-LSTM-TL architecture for feature extraction.



	ts	id.orig_h	duration	orig_bytes	resp_bytes
0	518258	1352	545	1	1
1	518257	1352	2579	13	170
2	518256	1352	43119	379	1
3	518260	1352	5256	26	13

	0	1	2	3	4
0	5.516609	3.980228	2.672321	-2.975865	1.323544
1	7.281296	3.897008	0.307558	-2.073243	-0.193115
2	13.443693	-1.404089	1.085177	8.634282	-2.093667
3	5.791917	5.093059	1.049947	-1.908625	0.488871

	Protocol	Flow Duration	Total Fwd Packets	Total Backward Packets	Fwd Packets Length Total	Bwd Packets Length Total
0	17	216631	6	0	2088.0	0.0
1	17	2	2	0	802.0	0.0
2	17	48	2	0	766.0	0.0
3	17	107319	4	0	1398.0	0.0

	0	1	2	3	4
0	1.562492	0.000000	0.000000	0.000000	0.311730
1	0.179162	0.000000	0.000000	0.000000	0.379284
2	0.000000	0.000000	0.000000	0.000000	0.000000
3	1.371853	0.000000	0.000000	0.000000	0.344466

Fig. 6. Data representation at each step in feature engineering.

VMware virtual machine on the same host, with 7.7 GB of allocated memory and two virtual CPU cores. The Ubuntu virtual machine hosted the client-server framework used for real-time simulation and resource utilization measurements.

5. Results & discussion

This section presents the experimental results obtained from both offline evaluation and simulation-based testing. Offline evaluation is conducted using a standard train-test split in a Jupyter environment, where model performance is assessed in terms of accuracy, precision, recall, F1-score, computational time, and 10-fold cross-validation. Following offline training and evaluation, the trained models are serialized and deployed within the proposed client-server framework for simulation-based testing, as described in Section 3. We first analyze offline performance to justify the feature-learning pipeline, followed by simulation-based real-time evaluation to assess deployment feasibility.

5.1. Offline testing results

In this section, we present the results of offline testing under different scenarios, including various cross-decomposition methods and the use of transfer learning, to identify the best approaches for testing in real-time scenarios.

5.2. Results without transfer learning approach

Table 4 presents experimental results using M-En data generated via the PLS canonical approach and without transfer learning. Notably, KNN and LGBM exhibit strong performance, achieving accuracies of 0.977 and 0.978, respectively, while RF closely follows with an accuracy of 0.973. The discrepancy suggests non-linearity in the PLS generated M-En data, favoring tree-based models like RF and LGBM over LR. Conversely, GNB yields the poorest performance, with an accuracy of 0.762.

Similarly, we compared PLS canonical with the CAA approach to demonstrate PLS canonical significance over other cross-decomposition methods for the M-En dataset generation. Table 5 presents the results obtained using the M-En dataset generated through the CCA approach. The performance of models is not as strong as with PLS canonical. RF performed well with an accuracy score of 0.973 using PLS, which decreased slightly to 0.969 using CCA. Similarly, LR and GNB also showed decreases in accuracy scores. While KNN remained consistent, LGBM performed slightly better using the PLS canonical approach compared to CCA.

Specifically, in terms of the confusion matrix showing incorrect and correct predictions, LGBM gives 440 incorrect predictions using PLS canonical and 452 incorrect predictions using CCA out of 20,000 test predictions, as shown in Fig. 7. As PLS canonical outperforms CCA, we exclusively employ PLS for further experiments.

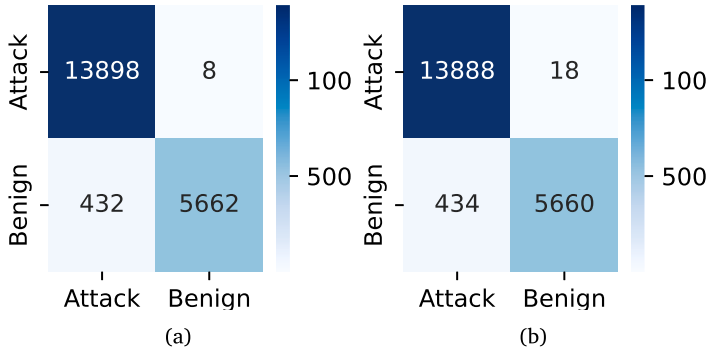


Fig. 7. Best performer confusion matrices; a) PLS & LGBM and b) CCA & LGBM.

Table 4
Results using PLS generated M-En dataset.

Model	Accuracy	Class	Precision	Recall	F1-Score
RF	0.973	Attack	0.97	0.99	0.98
		Benign	0.98	0.93	0.96
		Macro avg	0.98	0.96	0.97
LR	0.951	Attack	0.94	0.99	0.97
		Benign	0.98	0.85	0.91
		Macro avg	0.96	0.92	0.94
KNN	0.977	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
LGBM	0.978	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
GNB	0.762	Attack	0.77	0.94	0.85
		Benign	0.73	0.34	0.47
		Macro avg	0.75	0.64	0.66

Table 5
Results using CCA generated M-En dataset.

Model	Accuracy	Class	Precision	Recall	F1-Score
RF	0.969	Attack	0.97	0.98	0.98
		Benign	0.96	0.94	0.95
		Macro avg	0.97	0.96	0.96
LR	0.939	Attack	0.92	1.00	0.96
		Benign	0.99	0.81	0.89
		Macro avg	0.96	0.90	0.92
KNN	0.977	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
LGBM	0.977	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
GNB	0.729	Attack	0.72	0.99	0.84
		Benign	0.82	0.14	0.24
		Macro avg	0.77	0.56	0.54

5.2.1. Results with transfer learning approach

In this section, we present the performance of machine learning models using the M-En dataset generated by the PLS canonical and RNN-LSTM-TL approach. Table 6 shows the results, where features are extracted using RNN-LSTM-TL and utilized to train the machine learning models. Each model achieved notable results using transfer learning, as LGBM, LR, and GNB reached the highest accuracy scores of 0.979, 0.978, and 0.978, respectively. KNN also achieved significant results with an accuracy score of 0.977.

Table 6
Results using PLS generated M-En dataset and RNN-LSTM-TL.

Model	Accuracy	Class	Precision	Recall	F1-Score
RF	0.968	Attack	0.97	0.98	0.98
		Benign	0.96	0.94	0.95
		Macro avg	0.96	0.96	0.96
LR	0.978	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
KNN	0.977	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97
LGBM	0.979	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.97	0.97
GNB	0.978	Attack	0.97	1.00	0.98
		Benign	1.00	0.93	0.96
		Macro avg	0.98	0.96	0.97

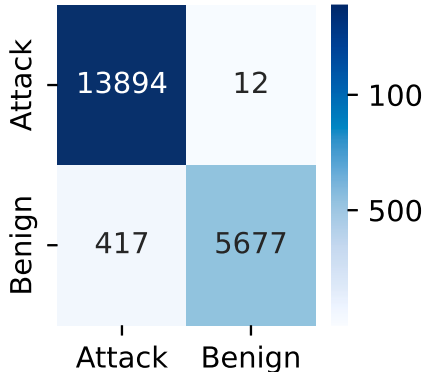


Fig. 8. LGBM confusion matrix using RNN-LSTM-TL.

Fig. 8 displays the confusion matrix for the best-performing LGBM model with the transfer learning approach, yielding only 429 wrong predictions, the lowest compared to other approaches. The models' performance improved significantly due to the linearly separable feature set generated by the RNN-LSTM-TL approach. Further analysis of the *false negative rate* (FNR) and *false positive rate* (FPR) shows that LGBM using RNN-LSTM-TL achieves a very low FNR of 0.086%, ensuring that almost all attacks are detected. The FPR of 6.8% is comparatively higher, indicating that some benign traffic is misclassified as malicious. However, this trade-off is acceptable in security contexts where missing attacks is far more critical than generating additional alerts.

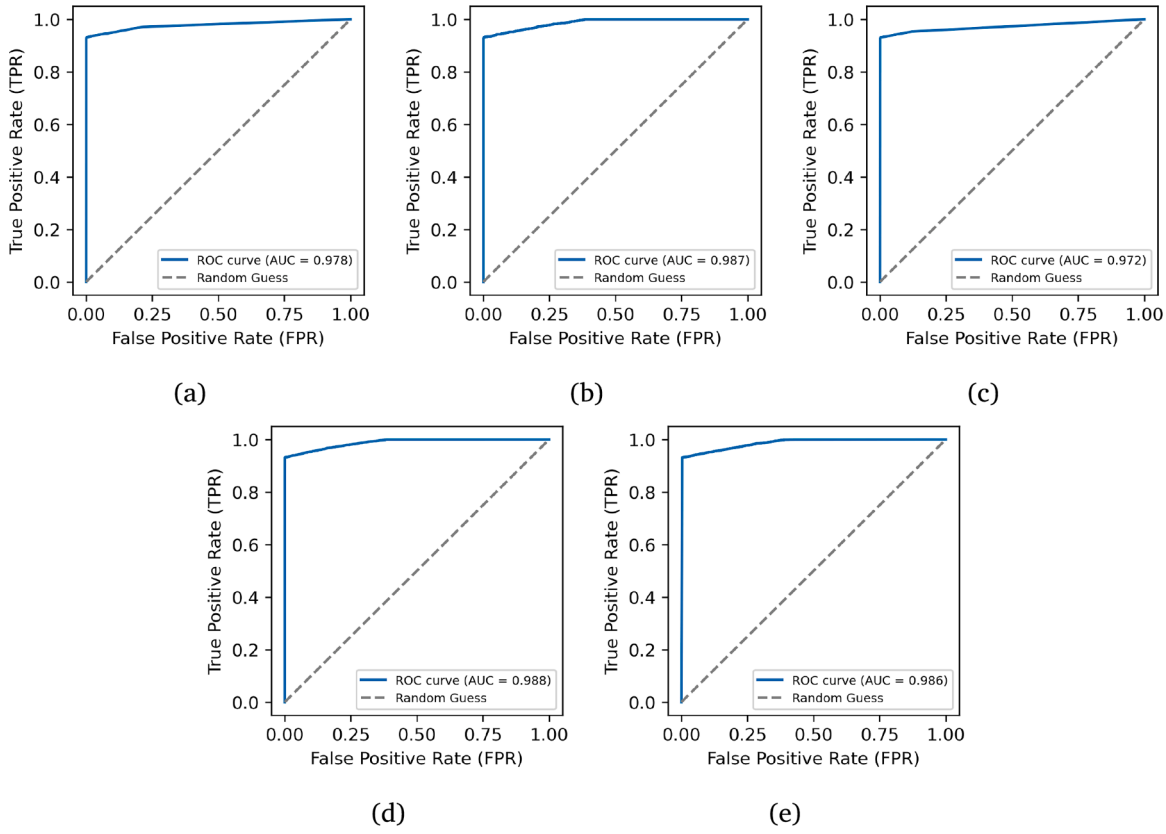


Fig. 9. ROC using RNN-LSTM-TL, (a) RF, (b) LR, (c) KNN, (d) LGBM, and (e) GNB.

Fig. 9 presents the ROC curves for the RF, LGBM, GNB, KNN, and LR, all trained on the temporal features extracted by the proposed RNN-LSTM-TL model. These classifiers achieve strong discrimination between benign and malicious traffic, with AUC values ranging from 0.972 to 0.988. At low false positive rates (FPR $<$ 0.05), tree-based models such as LGBM and RF maintain higher *true positive rates* (TPR) compared to linear and probabilistic approaches, reflecting their ability to exploit the richer temporal features. KNN also shows competitive performance, though slightly lower than LGBM. These results confirm that the RNN-LSTM-TL feature representations provide strong generalization across heterogeneous datasets, making them more suitable for real-time M-En scenarios where attack behaviors evolve over time.

Fig. 10 compares model accuracies across three configurations: PLS, CCA, and RNN-LSTM-TL. The RNN-LSTM-TL features consistently yield the highest accuracy across all models, showing the benefit of temporal feature extraction. Traditional PLS and CCA features show similar performance for RF, KNN, and LGBM, while LR and GNB experience more noticeable improvements when RNN-LSTM-TL is included. Notably, GNB shows the largest gain, increasing from around 0.73-0.76 to approximately 0.98, indicating that the transformed feature space substantially enhances model separability.

5.2.2. Results using K-fold cross validation approach

To further validate the performance of the proposed approach and baseline methods, we conducted experiments using a 10-fold cross-validation strategy, and the results are presented in Table 7. The performance of machine learning models drops with the 10-fold cross-validation approach compared to the train-test split approach, reflecting the increased variability across folds. Among all approaches, the RNN-LSTM-TL achieves the highest accuracy, with LR and GNB emerging as the top performers, each attaining a mean accuracy of 0.98 with a *standard deviation* (STD) of 0.01. In contrast, without RNN-LSTM-TL, performance is notably lower: LR achieves a mean accuracy of 0.94 using

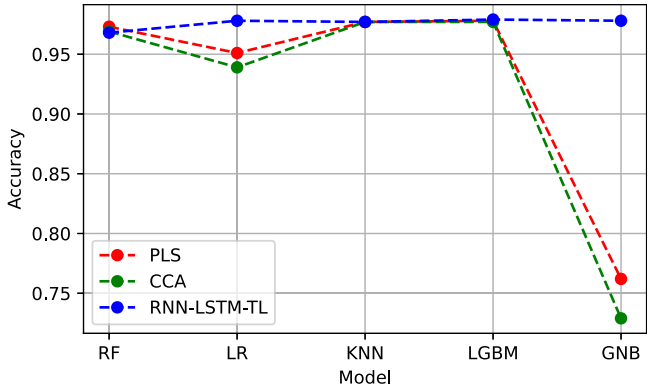


Fig. 10. Performance comparison between PLS, CCA, and RNN-LSTM-TL.

Table 7
Results using 10-Fold cross-validation approach.

Model	RNN-LSTM-TL		CCA		PLS	
	Accuracy	STD	Accuracy	STD	Accuracy	STD
RF	0.72	0.32	0.69	0.33	0.69	0.33
LR	0.98	0.01	0.89	0.08	0.94	0.04
KNN	0.72	0.32	0.71	0.33	0.70	0.34
LGBM	0.73	0.30	0.71	0.3	0.70	0.34
GNB	0.98	0.01	0.74	0.08	0.74	0.14

PLS, while CCA yields only 0.89 accuracy. The relatively large STD observed for some models (e.g., RF, KNN, and LGBM) reflects fold-wise variability due to the heterogeneous composition of the merged M-En dataset, where differences in traffic characteristics and attack distributions across folds increase model sensitivity.

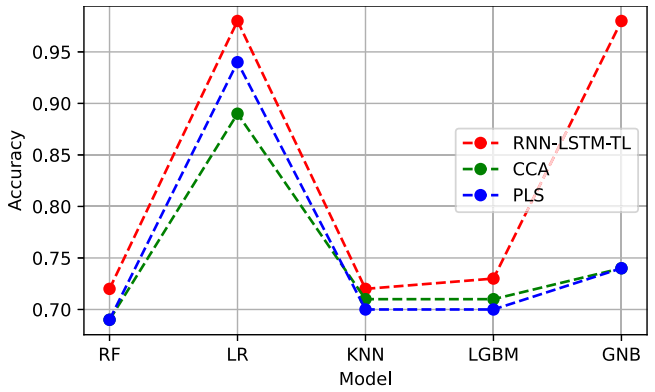


Fig. 11. Comparison between PLS, CCA, and RNN-LSTM-TL using 10-fold cross validation.

Fig. 11 shows the comparison of models' performance in different scenarios, such as PLS, CCA, and RNN-LSTM-TL approaches, using 10-fold cross-validation. The performance of the proposed RNN-LSTM-TL approach indicates significant improvement. LR and GNB perform very well with the proposed approach, achieving a mean accuracy score of 0.98, demonstrating the significance of our proposed approach in both train-test split methods and k-fold cross-validation.

5.3. Statistical significance analysis

To evaluate the effectiveness of ML models using the RNN-LSTM-TL approach compared to PLS and CCA, we perform a paired statistical t-test on the results from both standard and 10-fold cross-validation evaluations. The statistical t-test is a hypothesis test used to determine whether there is a significant difference between the means of two approaches' results [86]. It helps assess whether the observed differences are due to chance or an actual effect. The t-test calculates the t-value as:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}} \tag{11}$$

where $\bar{X}_1$ and $\bar{X}_2$ represent the sample means of the results of the two approaches, s_1^2 and s_2^2 are the variances of the results of each approach, and n_1 and n_2 denote the sample sizes of each approach. The t-test generates a p-value, which determines statistical significance. If the p-value is less than 0.05, the difference between the two approaches is considered statistically significant, leading to the rejection of the null hypothesis [87]. Conversely, if the p-value is greater than or equal to 0.05, there is no significant difference, and the null hypothesis is not rejected. The analysis we conduct with different confidence levels, ranging from the conventional 95% confidence level to a more relaxed 75% confidence level, allows us to assess the degree of statistical significance under varying degrees of certainty. At the conventional 95% confidence level ($\alpha = 0.05$), the p-values indicate that RNN-LSTM-TL could not exhibit a statistically significant improvement over PLS and CCA. However, when the confidence threshold is relaxed, RNN-LSTM-TL begins to show statistically significant differences, particularly compared to CCA. Table 8 presents the T-test results, including the T-statistics, p-values, and significance outcomes for each comparison evaluated at the *90%* ($\alpha = 0.10$), *85%* ($\alpha = 0.15$), *80%* ($\alpha = 0.20$), and *75%* ($\alpha = 0.25$) confidence levels.

The results show that at an 85% and 80% confidence level, ML models with RNN-LSTM-TL exhibit statistically significant improvements over CCA and PLS. At a 75% confidence level, the performance of RNN-LSTM-TL consistently outperforms both methods, suggesting that when a higher level of uncertainty is acceptable, RNN-LSTM-TL can be considered a more effective approach. Notably, the performance of the models under strict validation using k-fold shows more promising results compared to other methods, highlighting the significance of RNN-LSTM-TL in M-En networks for malicious traffic detection.

Table 8
T-Test results for RNN-LSTM-TL vs. PLS and CCA.

Comparison	K-Fold	T-Statistic	P-Value	Null Hypothesis Reject/Accept			
				90%	85%	80%	75%
RNN-LSTM-TL vs. PLS	No	1.709	0.1625	No	No	Yes	Yes
RNN-LSTM-TL vs. CCA	No	1.821	0.1426	No	Yes	Yes	Yes
RNN-LSTM-TL vs. PLS	Yes	1.709	0.1625	No	No	Yes	Yes
RNN-LSTM-TL vs. CCA	Yes	1.821	0.1426	No	Yes	Yes	Yes

Table 9
Computational time.

Model	CCA	PLS	RNN-LSTM-TL
RF	3 min 5 s	3 min 1 s	32.3 s
LR	1.63 s	1.37 s	521 ms
KNN	3.13 s	2.81 s	4.62 s
LGBM	2.03 s	2.45 s	2.13 s
GNB	1.07 s	1.04 s	135 ms

5.4. Computational time & feature space analysis

The performance of machine learning models is also assessed in terms of combined training and testing time, measured in *milliseconds* (ms) and *seconds* (s). Table 9 shows the computational time of models using CCA, PLS, and RNN-LSTM-TL approaches, revealing that models using RNN-LSTM-TL have lower computational time compared to the others. RNN-LSTM-TL extracts robust, high-quality features that encapsulate patterns and representations learned from extensive prior training. This quality enables machine learning models to converge faster and achieve better performance with less computational effort. LR and GNB have the lowest computational costs of 521 ms and 135 ms, respectively.

Fig. 12 illustrates the feature space for all approaches. Fig. 12c displays the feature space using the PLS canonical approach, showing that data samples have less overlap compared to the CCA samples in Fig. 12b. However, the feature space for RNN-LSTM-TL is more linearly separable compared to PLS canonical and CCA, as shown in Fig. 12c. This higher degree of linear separability simplifies the decision boundaries between benign and malicious samples, enabling linear classifiers, such as LR, to perform more effectively. In essence, when the feature space is well-structured and nearly linearly separable, LR can achieve strong discrimination with minimal parameter complexity, benefiting from the robust temporal and contextual feature extraction achieved by RNN-LSTM-TL.

To further demonstrate the significance of our proposed approach, RNN-LSTM-TL, we evaluated it using different architectures by varying the number of neurons and layers. These architectural changes did not lead to significant improvements in accuracy. While increasing the number of neurons introduced additional complexity and higher computational cost, it did not yield substantially better performance. On the other hand, reducing the number of neurons resulted in slightly lower accuracy but significantly reduced computational cost. Our chosen architecture offers a balanced trade-off, achieving both high accuracy and optimal computational efficiency. Three alternative architectures, similar to RNN-LSTM-TL but varying in neuron count and layer configuration, are summarized in Table 10.

Table 11 shows the performance comparison of the different architectures. The performance of A1 is optimal and very close to our proposed RNN-LSTM-TL, though it is slightly more expensive in terms of training time. However, it still maintains high accuracy. A2, which uses more neurons, results in a higher computational cost without a significant improvement in accuracy. On the other hand, reducing the number of neurons leads to a slight decrease in accuracy but also reduces computational cost. Our proposed RNN-LSTM-TL, as discussed in Sec-

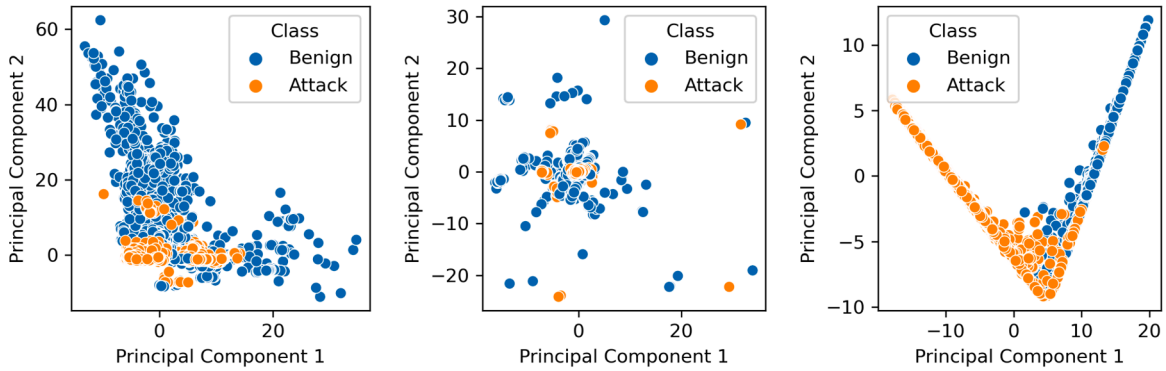


Fig. 12. Feature space using a) PLS, b) CCA, and c) RNN-LSTM-TL.

Table 10
RNN-LSTM-TL architectures variants.

Architecture (A1)	Architecture (A2)	Architecture (A3)
SimpleRNN (256)	SimpleRNN (256)	SimpleRNN (32)
LSTM (128)	LSTM (256)	LSTM (32)
Dropout (0.5)	Dropout (0.2)	Dropout (0.8)
Dense (64, relu)	Dense (64, relu)	Dense (16, relu)
Dropout (0.5)	Dropout (0.2)	Dropout (0.8)
Dense (32, relu)	Dense (32, relu)	Dense (8, relu)
Dense (1, sigmoid)	Dense (1, sigmoid)	Dense (1, sigmoid)

Table 11
Results using different architectures of RNN-LSTM-TL.

Model	A1	A2	A3	RNN-LSTM-TL
RF	0.966	0.966	0.960	0.968
LR	0.973	0.976	0.971	0.978
KNN	0.972	0.972	0.967	0.977
LGBM	0.977	0.978	0.971	0.979
GNB	0.970	0.976	0.971	0.978
Training Time	38.3	46.6	31.1	34.8

tion 4.2, provides the best balance between accuracy and computational efficiency. All our proposed RNN-LSTM-TL architectures are based on the same approach; therefore, there is no significant difference in accuracy or computational cost.

5.5. Real-time simulation results

In this section, we present the results from the simulation-based real-time evaluation of the proposed malicious traffic detection framework. Pre-trained models obtained from offline training, including the PLS canonical transformation, the RNN-LSTM-based transfer learning feature extractor, and various machine learning classifiers, are utilized without further retraining. Traffic is captured on the client side and forwarded to the server for batch-based inference and decision-making. In the simulation-based real-time evaluation, traffic is replayed from labeled benchmark datasets (IoT23 and CICDDoS2019), and ground-truth labels are therefore known *a priori*. Each evaluation sample corresponds to a single network packet with an associated label. Packets are buffered into batches of 400-1200 packets, and inference is performed at batch granularity. Accuracy is computed at the packet level by comparing predicted packet labels against ground-truth labels, and the reported real-time accuracy values represent mean packet-level accuracy aggregated across all processed batches. We evaluate performance based on accuracy, memory usage, and CPU usage. Fig. 13 shows memory and CPU usage over different batches. Memory usage fluctuates based on the number of packets received by the server; larger traffic volumes require more memory. In contrast, CPU usage remains relatively consistent, demonstrating the efficiency of the proposed approach and its low resource consumption capabilities.

Table 12
Real-time simulation mean scores.

Model	Accuracy	Time (Sec)	Memory (MB)	CPU%
RF	0.69	0.68	5.63	1.64
LR	0.71	0.59	2.96	0.99
KNN	0.60	1.06	3.26	23.72
LGBM	0.67	0.54	1.4	10.89
GNB	0.65	0.58	2.80	0.90

Further, we evaluate each model's performance in terms of mean packet-level accuracy aggregated across batches, mean per-batch prediction time, mean memory usage, and mean CPU utilization measured at the server side. For each incoming batch, wall-clock prediction time is recorded from batch reception to completion of feature extraction and inference, while CPU and memory usage are measured at the process level and averaged across all evaluated batches. The reported CPU utilization reflects the average proportion of system CPU resources consumed during batch processing, and differences across models are attributable to their computational characteristics. The results show that the LR model achieves the best balance between accuracy and computational efficiency during simulation-based evaluation, attaining a mean packet-level accuracy of 0.71 compared to 0.69 for the RF model. In addition to its competitive accuracy, LR exhibits moderate per-batch prediction time (mean of 0.59 seconds) and low memory consumption (mean of 2.96 MB), making it well-suited for deployment-oriented, batch-based gateway scenarios. Table 12 presents the simulation-based real-time performance metrics for all evaluated models.

In addition, Fig. 14a illustrates a snapshot of the real-time simulation, capturing key performance metrics for two example batches. In the first batch, the server received 1018 packets, achieving an accuracy of 80.97% in 0.42 seconds. In the subsequent batch, the server processed 432 packets with a recorded accuracy of 72.42% in 0.30 seconds. Fig. 14b shows the comparison between offline and real-time testing and shows that LR achieves the highest accuracy in both offline (0.98) and real-time (0.71) settings, while also being resource-efficient with low prediction time and memory usage. In contrast, KNN has the highest CPU usage and slower prediction times, making it less suitable for real-time detection.

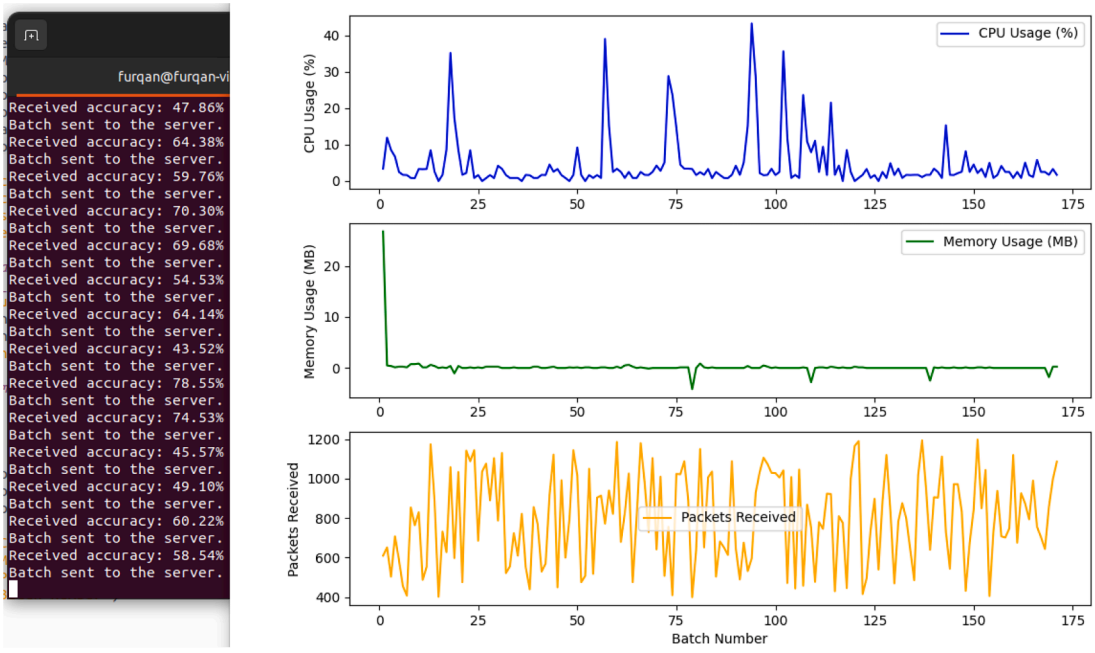


Fig. 13. Real-time prediction snapshot for memory and CPU usage.

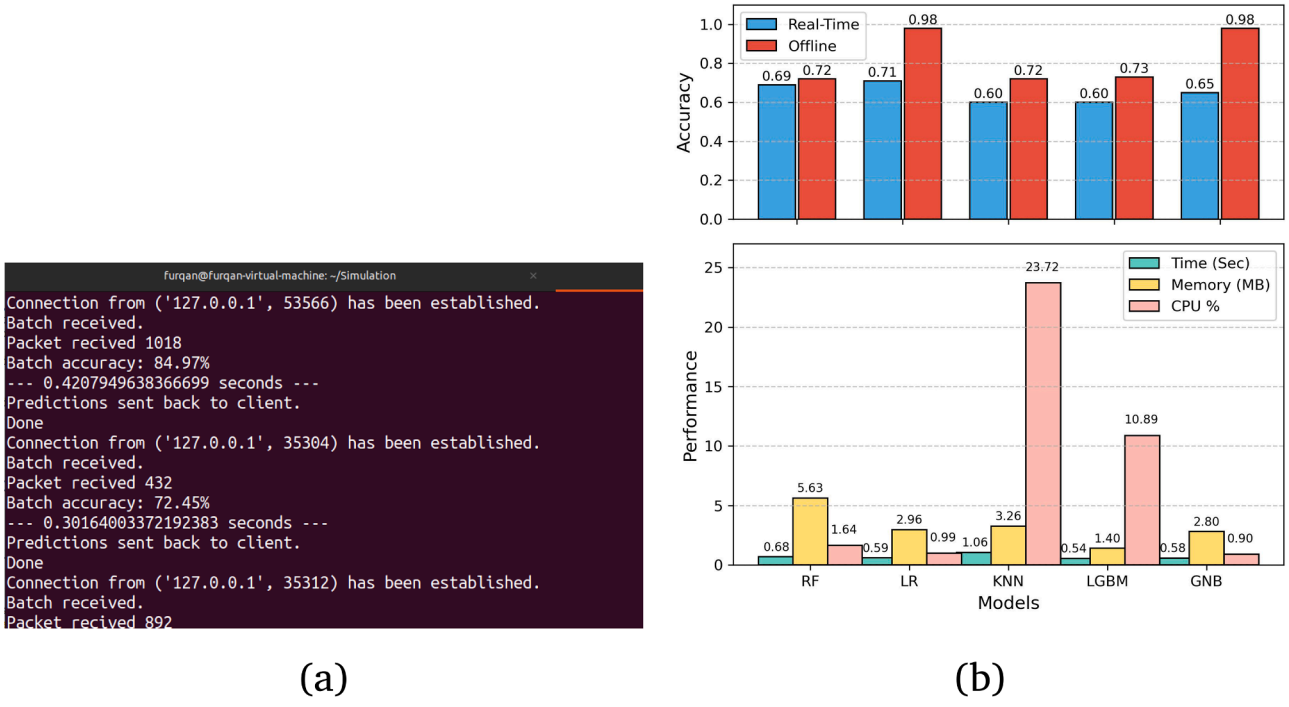


Fig. 14. Real-Time simulation results, a) LR real-time prediction snapshot and b) Performance comparison of models in terms of offline and real-time testing.

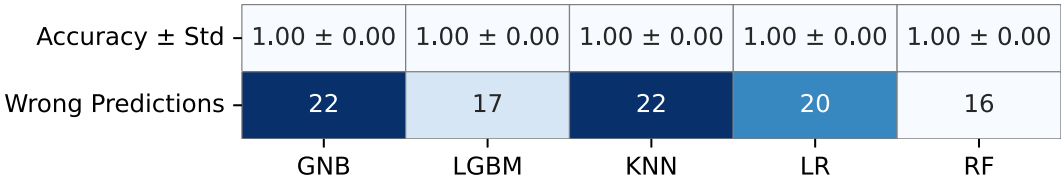


Fig. 15. Offline testing results of the proposed approach on the realistic M-En dataset.

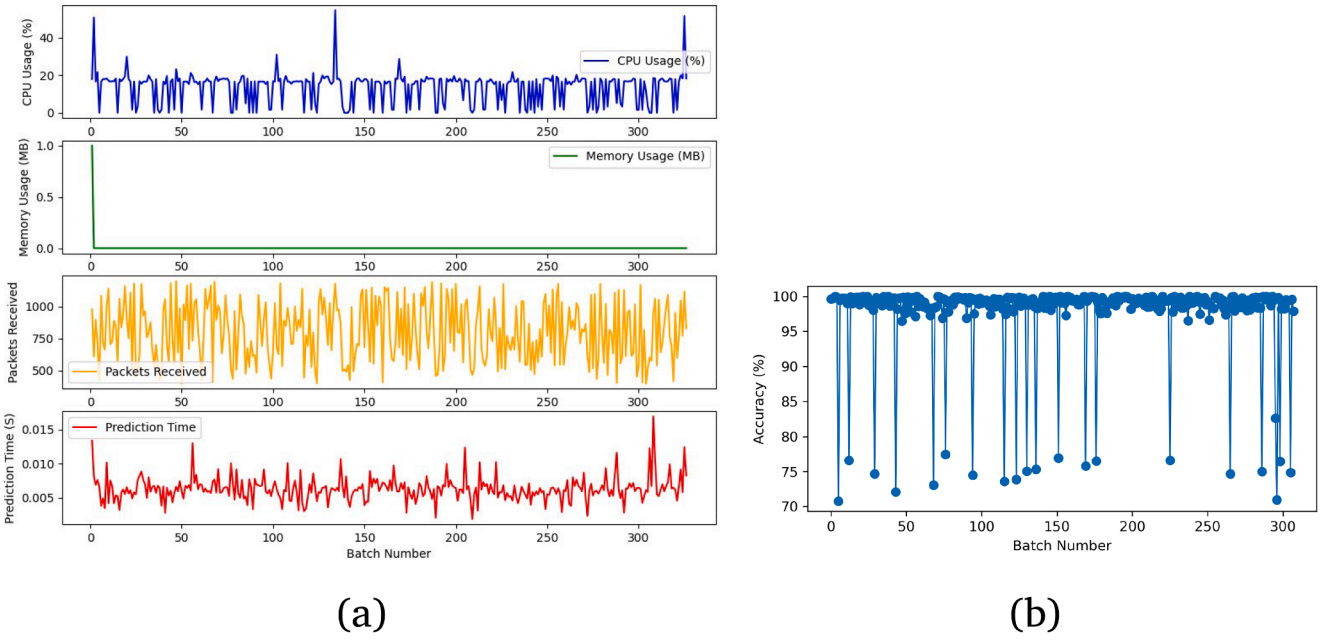


Fig. 16. Real-time simulation results for realistic M-En data, a) LR performance metrics over different batches and b) LR accuracy over different batches.

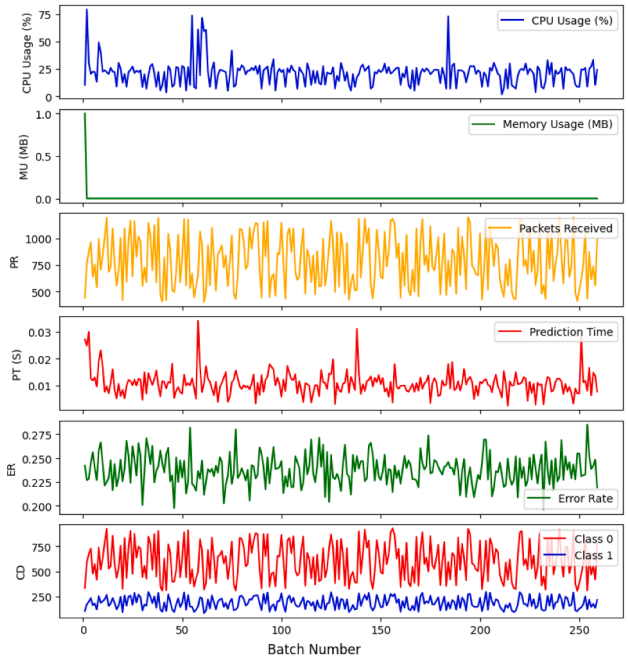


Fig. 17. Batch-level diagnostic analysis during simulation-based evaluation.

5.6. Results using realistic M-En dataset

To provide a deeper evaluation, we also assess the proposed approach on our realistic M-En dataset [88]. This newly built dataset consists of real M-En traffic data, which is a combination of IoT and Tr. IP-based network traffic. It is distinct from the constructed M-En dataset used earlier in this study, which was generated by merging IoT23 and CICDDoS2019 using PLS canonical analysis. Since this realistic M-En dataset already contains merged features from both environments, there is no need to apply the PLS merging approach. After selecting the important features using the ETC, we passed them to the RNN-LSTM-TL model and subsequently applied various ML models. The results of our approach in offline testing are shown in Fig. 15. All models achieved significant performance when trained on 80,000 samples and tested on 20,000 samples. The evaluation is primarily based on the number of incorrect predictions. Among the models, RF and LGBM performed well with fewer misclassifications. However, the computational cost of LR was notably lower compared to others, with a CPU time of 625 ms and a wall time of 619 ms during offline testing, making it a suitable choice for real-time deployment.

The LR model was then evaluated in a real-time simulation setup on an Ubuntu VM. The model demonstrated significant performance on new incoming data, evaluated over more than 300 data batches. The performance metrics are illustrated in Fig. 16a. The computational cost remained exceptionally low, while the accuracy reached as high as 1.00, with an average accuracy of 0.975 and a standard deviation of ±6.14. The variation in LR performance across different batches is shown in Fig. 16b. The real-time accuracy difference between the realistic M-En traffic and our generated M-En dataset is mainly due to differences in their training data. The realistic M-En dataset was captured in the same environment and has a unified feature set covering both IoT and Tr. IP-based network traffic. This consistent feature space helps the model learn more effectively. In addition, the dataset contains only a few attack categories, which reduces complexity. During real-time testing, the traffic patterns are also similar to those seen in training, which explains its higher performance.

In contrast, the generated M-En dataset was collected in different environments, with heterogeneous feature sets and many diverse attack categories compared to the M-En dataset. Although our PLS canonical approach and RNN-LSTM-TL generate a robust unified feature representation that improves accuracy, real-time testing remains more challenging due to the diversity of attack types and the increased complexity of the feature space. These results also highlight the importance of a unified feature set when collecting data for M-En, as it is crucial for building effective malicious traffic detection systems.

5.6.1. Latency and throughput analysis

In our experimental setup using the representative M-En dataset, we processed 309 batches with an average of 844.17 samples per batch, resulting in an estimated total of $N_{\text{total}} \approx 260{,}848$ samples. The average batch execution time was $T_{\text{avg_batch}} = 0.23$ seconds, yielding a total pro-

cessing time of $T_{\text{total}} = 71.07$ seconds. This corresponds to an average processing time per packet of $T_{\text{avg_packet}} = T_{\text{total}}/N_{\text{total}} \approx 2.72 \times 10^{-4}$ seconds (approximately 272 μs), enabling a throughput of approximately 3676 packets per second. This throughput estimate is derived from aggregate packet processing and serves as a deployment-oriented feasibility indicator under simulated gateway conditions. The system maintains an average CPU utilization below 20%, indicating sustainable batch-based inference with moderate computational overhead. For reference, the CIC-IDS-2017 dataset [89] contains approximately 10.9 million packets captured over eight hours (about 380 packets/s), providing an empirical context for the traffic characteristics of widely used intrusion detection benchmarks relative to which the proposed framework demonstrates substantially higher processing capacity.

5.7. Comparison with other studies

In this section, we compare our proposed approach with other studies that have worked in the domain of malicious traffic detection. Currently, to the best of our knowledge, prior studies in M-En networks have largely focused on offline evaluation, with limited attention to deployment-oriented, batch-based simulation at the gateway level. However, for a fair comparison, we deployed the methods from other studies within our setup and tested them under the same conditions using our generated M-En dataset.

We selected studies based on recency and the use of packet-level features in their experiments, which are similar to those in our study. For example, Rustam et al. [5] proposed an M-En MTDS and worked on generating an M-En dataset. They created the M-En dataset by combining multiple datasets that include packet-level features, as in our study. However, they used outdated attack pattern datasets such as UNSWNB15 and IoTID20. They used an RF classifier, with ETC and NN for feature fusion, and applied MFO to optimize the system. We deployed their FAMTDS in our scenario and tested it with our recently generated M-En dataset, which consists of the latest attack patterns. Similarly, the approach in [90] utilized a hybrid CNN-GRU model combined with an *Autoencoder* (AE) for feature extraction and alignment, aimed at efficient malicious traffic detection. In their original work, they used network traffic data from IIoT systems (e.g., Edge-IIoTSet, WUSTL-IIoT-2021, X-IIoTID), which are similar to our dataset that also consists of packet-level features. To reproduce their approach for our M-En dataset, we used their AE to align feature spaces and then merged the datasets to generate the M-En dataset before applying the CNN-GRU. While the original study used Bayesian optimization, we tested their approach without optimization in our setup.

Another study [91] proposed a unique method by first converting the CIC-Bell-S2021 dataset samples into images and then applying pre-trained models to achieve significant results. Their dataset also consisted of packet-level features, which they converted into images as part of their approach to improve performance. We adapted their method by using the same AE architecture to generate 4096-dimensional representations for each dataset in M-En. After combining the datasets, we converted them into 64x64 images and used ResNet-50 for classification. Rustam et al. [1] used PSO-D-SEM method, employed ETC to select an equal number of features for each dataset, generating an M-En dataset, followed by ETC for malicious traffic prediction. Their approach used UNSWNB15, IoTID20, and InSN datasets, all of which are packet-level datasets. Similarly, [92] proposed a *robust transformer-based intrusion detection system* (RTIDS), a hierarchical Transformer-based IDS using multi-head self-attention and positional encoding to capture complex temporal and contextual traffic patterns. We apply RTIDS on PLS features and evaluate it only in offline mode due to its high computational and memory demands. Additionally, the study [93] proposed a *Transformer Neural Network-Based Intrusion Detection System* (TNN-IDS) that applies multi-head self-attention to flow features, with Conv1D feed-forward layers, residual connections, and layer normalization forming a lightweight Transformer encoder block for tabular network data.

We evaluate all approaches in both real-time and offline settings to ensure a fair comparison. Our proposed MTDS shows significant improvements in accuracy and efficiency relative to prior work. While methods such as Rustam et al. [5] and Rustam and Jurcut [1] achieve strong offline accuracy, their computational cost is considerably higher because their optimization procedures require substantial training time to converge. The method in [91] also incurs significantly higher real-time prediction cost, approximately three times slower than ours, due to its multi-stage pipeline that first transforms input through an autoencoder, then converts it into an image, and finally performs deep CNN inference. In contrast, Table 13 shows that our MTDS achieves a more favorable balance between complexity and throughput, outperforming previous approaches in both real-time and offline evaluations. This advantage arises from our feature engineering strategy based on ETC and RNN-LSTM-TL, coupled with PLS-based cross-decomposition, which effectively captures correlation across heterogeneous IoT and Tr. IP-based datasets and improves detection performance in the M-En environment.

5.8. Discussion

Developing a unified M-En MTDS offers significant benefits, such as cost savings, improved computational efficiency, and reduced management overhead. However, building an effective unified MTDS requires a well-designed M-En dataset. In this study, we generate an efficient M-En dataset using PLS, which identifies relationships between features before fusing them into a unified feature space, which is limited in the previous studies on M-En dataset generation approaches [1,5]. Traditional MTDS typically focuses on either IoT or Tr. IP-based (non-IoT) network traffic, such as ToN-IoT [56] and UNSWNB15 [25]. When machine learning models are trained on just one type of network traffic, such as IoT or Tr. IP-based traffic often performs poorly when tested on the other type, as shown in Table 14. This performance gap is due to the inherent differences in traffic patterns and protocols between IoT and Tr. IP-based networks. IoT devices typically produce low-bandwidth, continuous traffic using lightweight protocols [94], whereas Tr. IP-based networks involve more varied, higher-bandwidth communication with complex protocols [95]. Table 14 shows that all models trained on IoT datasets perform poorly on Tr. IP-based datasets, with performance scores below 0.50 in all cases, underscore a significant performance drop. This decline highlights the challenges of using models trained on isolated IoT data when applied to Tr. IP-based network environments. Given these limitations, building a unified MTDS solely from individual datasets is challenging, which motivated us to develop the M-En dataset specifically for these experiments.

From a feature engineering perspective, the PLS technique effectively generates a feature space by leveraging correlations between IoT and Tr. IP-based datasets, unlike PCA and Chi-squared, which overlook such relationships [1,5]. Combined with the RNN-LSTM-TL architecture, which captures sequential patterns and temporal dependencies, this approach produces a rich feature space that improves machine learning model performance.

Regarding the difference in performance between simulation-based and offline evaluations, we also analyze batch-level system behavior during streaming evaluation to better understand this variation. Fig. 17 illustrates CPU usage, memory usage (MU), packets received (PR) per batch, prediction time (PT), error rate (ER), and class distribution (CD) across batches. The results show that the benign-to-malicious ratio varies across batches, which can influence prediction performance during deployment-oriented evaluation. Unlike offline experiments, where class distributions are relatively stable, streaming traffic may produce temporary class imbalance, leading to fluctuations in batch-level error rates. Since simulation batches are formed from streamed traffic, their class composition can differ across batches, which may introduce minor run-to-run variability in aggregated performance metrics. To maintain a consistent evaluation, we report the mean accuracy aggregated across all processed batches under a fixed evaluation configuration, while the di-

Table 13
Comparison with existing methods.

Ref.	Year	Methods	RTA	OA	$\mathcal{O}$	Γ
[92]	2022	RTIDS	–	0.970	↑	–
[93]	2023	TNN-IDS	0.569	0.955	↑	$\Gamma_{Uni/Bi}$: 10.7–128 s/s Γ_{packet} : 0.10-0.77 s/s ○↓
[1]	2024	ETC + PSO-D-SEM	0.688	0.929	→	362,903 s/s ○↑
[5]	2024	FAMTDS (ETC + RF + NN + MFO)	0.578	0.935	↑	15,600 s/s ○
[90]	2024	AE + CNN-GRU	0.680	0.961	→	–
[91]	2024	ResNet-50	0.371	0.815	↑	○↓
Proposed MTDS	2025	PLS + RNN-LSTM-TL + ML	0.71 (LR)	0.979 (LGBM)	↓	Offline: 153,846 s/s ○↑ Real-time: 3,676 s/s ○

Ref.– Reference; **RTA**– Real-time Accuracy; **OA**— Offline Accuracy; $\mathcal{O}$— Method Complexity; Γ— Throughput; **s/s**— samples/sec; —— **Symbols**: ↑ High Complexity, → Medium Complexity, ↓ Low Complexity, ○↑ High Throughput, ○ Medium Throughput, ○↓ Low Throughput.

Table 14
ML model performance in cross-network scenarios: IoT vs. Tr. IP-based training.

Model	Accuracy	Precision	Recall	F1 Score
Trained on IoT and Tested on Tr. IP-based Network				
RF	0.06	0.03	0.22	0.06
LR	0.06	0.05	0.21	0.06
KNN	0.20	0.58	0.53	0.20
LGBM	0.11	0.22	0.27	0.11
GNB	0.15	0.07	0.50	0.13
Trained on Tr. IP-based and Tested on IoT Network				
RF	0.46	0.25	0.50	0.32
LR	0.41	0.22	0.44	0.29
KNN	0.07	0.06	0.08	0.07
LGBM	0.06	0.05	0.07	0.06
GNB	0.46	0.23	0.50	0.32

agnostic figure provides insight into the batch-level dynamics observed during simulation.

Further, most studies proposing MTDS solutions conduct testing exclusively in offline settings, which limits their reliability in real-time scenarios [18,32]. Offline testing typically involves evaluating model performance using subsets of the same dataset used for training, rather than truly new or varied data. In contrast, a real-time MTDS must make decisions on potentially unseen data or on traffic patterns that differ slightly from the training set, which often results in a drop in detection accuracy when models are deployed in live environments [17,43]. By incorporating transfer learning into our methodology, we enhance the MTDS models' responsiveness to these real-time variations, thereby improving detection accuracy and operational efficiency when applied to dynamic, live network environments.

6. Conclusion & future work

This study conducted experiments to develop a simulation-based real-time unified approach for protecting M-En networks from malicious actors. An M-En dataset was generated using the PLS canonical approach, while RNN-LSTM-TL was employed for feature extraction. An Ubuntu-based client-server framework was implemented to create a real-time network simulation for testing the MTDS. The findings show that cross-decomposition techniques can effectively merge datasets to generate an M-En dataset with lower computational costs compared to state-of-the-art methods. Specifically, the PLS canonical approach demonstrates greater efficiency with both categorical and numerical features compared to other cross-decomposition methods. Additionally, the transfer learning approach, particularly RNN-LSTM-TL, effectively captures dependencies and sequences in network packets, enabling the generation of linearly separable feature sets. The LR model shows promising performance with linearly separable data, achieving a high accuracy of approximately 0.98 and a low computational time of 521 ms in offline testing. Our study also concludes that offline testing alone is insufficient for validating the efficiency of AI-based models, as accuracy can drop significantly in real-time environments. For example, the MTDS achieved the highest accuracy of 0.98 in offline testing but only 0.71 in real-time simulation, highlighting the challenges faced when new data arrives for testing in dynamic environments.

Limitation & Future Work: This study has certain limitations that we plan to address in future work. First, while our real-time evaluation uses a localhost client-server simulation to provide controlled and repeatable testing, future work will extend the evaluation to a small-scale hardware or cloud testbed to support testing in more realistic conditions. We will incorporate continuous learning to adapt dynamically to new traffic patterns and evaluate latency under more realistic conditions using pcap-replay with background noise. We also plan to strengthen secure model deployment by investigating defenses against adversarial attacks, data poisoning, spoofing, and evasion, including adversarial retraining and ensemble hardening to improve MTDS robustness against evasion and reduce susceptibility to adversarial manipulation. Further, we will extend the M-En dataset to incorporate ICS, SDN, and PLC environments, enabling broader protocol-level coverage and truly unified malicious traffic detection. Finally, advanced techniques and human-in-the-loop approaches will be explored to improve accuracy in complex cases.

CRediT authorship contribution statement

Furqan Rustam: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Anca Delia Jurcut:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Formal analysis.

Dataset & Code

The data and code are available in a public repository: https://github.com/furqanrustam/M-En-PLS-and-RNN-LSTM-Ubuntu-Simulation.

Data availability

The data and code are available in a public repository: https://github.com/furqanrustam/M-En-PLS-and-RNN-LSTM-Ubuntu-Simulation.

Declaration of competing interest

The author declares that there are no conflicts of interest. Additionally, no experiments involving human or animal subjects were performed in this study.

Acknowledgment

This work was funded by the School of Computer Science at University College Dublin (UCD) and supported by the European Cooperation in Science and Technology (COST) Actions CA22168, CA24169, and CA22104.

References

[1] F. Rustam, A.D. Jurcut, Malicious traffic detection in multi-environment networks using novel S-DATE and PSO-D-SEM approaches, Computers Secur. 136 (2024) 103564.
[2] J. Wang, Z. Yan, H. Wang, T. Li, W. Pedrycz, A survey on trust models in heterogeneous networks, IEEE Commun. Surv. Tutor. 24 (4) (2022) 2127–2162.
[3] F. Rustam, A.D. Jurcut, Malicious traffic detection in multi-environment networks using novel S-DATE and PSO-D-SEM approaches, Computers Secur. 136 (2024) 103564. https://doi.org/10.1016/j.cose.2023.103564
[4] D. Blog, What is Network Segmentation?, 2023, Accessed: 2023-11-08, https://www.dashlane.com/blog/what-is-network-segmentation.
[5] F. Rustam, W. Aljedaani, M.S. Elsayed, A.D. Jurcut, FAMTDS: a novel MFO-based fully automated malicious traffic detection system for multi-environment networks, Comput. Networks 251 (2024) 110603.
[6] D. Dzung, M. Naedele, T.P. Von Hoff, M. Crevatin, Security for industrial communication systems, Proc. IEEE 93 (6) (2005) 1152–1177.
[7] S. Vitturi, C. Zunino, T. Sauter, Industrial communication systems and their future challenges: next-generation ethernet, IIoT, and 5G, Proc. IEEE 107 (6) (2019) 944–961.
[8] S. Scanzio, L. Wisniewski, P. Gaj, Heterogeneous and dependable networks in industry–A survey, Comput. Ind. 125 (2021) 103388.
[9] F. Rustam, P.S. Ranaweera, A.D. Jurcut, AI on the defensive and offensive: securing multi-Environment networks from AI agents, in: IEEE ICC 2024, 2024.
[10] U. Zukaib, X. Cui, C. Zheng, D. Liang, S.U. Din, Meta-fed IDS: meta-learning and federated learning based fog-cloud approach to detect known and zero-day cyber attacks in ioMT networks, J. Parallel Distrib. Comput. (2024) 104934.
[11] P.L. Indrasiri, E. Lee, V. Rupapara, F. Rustam, I. Ashraf, Malicious traffic detection in iot and local networks using stacked ensemble classifier, Computers Mater. Continua 71 (1) (2022) 489–515.
[12] V. Kumar, A.A. Laghari, S. Karim, M. Shakir, A.A. Brohi, Comparison of fog computing & cloud computing, Int. J. Math. Sci. Comput. 1 (2019) 31–41.
[13] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, R. Boutaba, Network function virtualization: state-of-the-art and research challenges, IEEE Commun. Surv. Tutor. 18 (1) (2016) 236–262. https://doi.org/10.1109/COMST.2015.2477041
[14] S.A.H. Mohsan, H. Amjad, A comprehensive survey on hybrid wireless networks: practical considerations, challenges, applications and research directions, Opt. Quantum Electron. 53 (9) (2021) 523.
[15] R. Walsh, L. Xu, T. Paila, Hybrid networks–a step beyond 3G, in: Third International Symposium on Wireless Personal Multimedia Communications (WPMC'00), Citeseer, 2000, pp. 109–114.
[16] J. Hesford, D. Cheng, A. Wan, L. Huynh, S. Kim, H. Kim, J.B. Hong, Expectations versus reality: evaluating intrusion detection systems in practice, arxiv preprint, (2024). arXiv:2403.17458
[17] P. Mell, V. Hu, R. Lippmann, J. Haines, M. Zissman, An overview of issues in testing intrusion detection systems, National Institute of Standards and Technology, NIST Interagency Report, NISTIR 7007 (2003). https://doi.org/10.6028/NIST.IR.7007
[18] S.P. Thirimanne, L. Jayawardana, L. Yasakethu, P. Liyanaarachchi, C. Hewage, Deep neural network based real-time intrusion detection system, SN Computer Sci. 3 (2) (2022) 145.
[19] R. Rosipal, N. Krämer, Overview and recent advances in partial least squares, in: International Statistical and Optimization Perspectives Workshop" Subspace, Latent Structure and Feature Selection", Springer, 2005, pp. 34–51.
[20] S.M. Al-Selwi, M.F. Hassan, S.J. Abdulkadir, A. Muneer, E.H. Sumiea, A. Alqushaibi, M.G. Ragab, RNN-LSTM: From applications to modeling techniques and beyond–systematic review, J. King Saud Univ.-Computer Inf. Sci. (2024) 102068.
[21] S. Ali, R. Alauldeen, A. Ruaa, What is client-server system: architecture, issues and challenge of client-server system, HBRP Publication 2 (1) (2020) 1–6.
[22] F. Marini, B. Walczak, Particle swarm optimization (PSO). A tutorial, Chemometr. Intell. Laborat. Syst. 149 (2015) 153–165.
[23] F. Rustam, A.D. Jurcut, W. Aljedaani, I. Ashraf, Securing multi-environment networks using versatile synthetic data augmentation technique and machine learning algorithms, in: 2023 20th Annual International Conference on Privacy, Security and Trust (PST), 2023, pp. 1–10. https://doi.org/10.1109/PST58708.2023.10320203
[24] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in iot networks, in: Canadian Conference on Artificial Intelligence, Springer, 2020, pp. 508–520.
[25] N. Moustafa, J. Slay, UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: 2015 Military Communications and Information Systems Conference (MilCIS), IEEE, 2015, pp. 1–6.
[26] L. Xu, M. Skoularidou, A. Cuesta-Infante, K. Veeramachaneni, Modeling tabular data using conditional gan, Adv. Neural Inf. Process. Syst. 32 (2019).
[27] M. Nivedhidha, M.P. Ramkumar, E.S. GSR, et al., CopulaGAN boosted random forest based network intrusion detection system for hospital network infrastructure, in: 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), IEEE, 2023, pp. 1–7.
[28] A. Kiran, S.S. Kumar, A comparative analysis of gan and vae based synthetic data generators for high dimensional, imbalanced tabular data, in: 2023 2nd International Conference for Innovation in Technology (INOCON), IEEE, 2023, pp. 1–6.
[29] M. Shehab, L. Abualigah, H. Al Hamad, H. Alabool, M. Alshinwan, A.M. Khasawneh, Moth–flame optimization algorithm: variants and applications, Neural Comput. Appl. 32 (14) (2020) 9859–9884.
[30] F. Rustam, R. Shafique, S.K. Posa, A.D. Jurcut, Malicious traffic detection in multi-environment network using dual-Data trained lightGBM approach, in: 2024 IEEE 21st International Conference on Mobile Ad-Hoc and Smart Systems (MASS), IEEE Computer Society, Los Alamitos, CA, USA, 2024, pp. 598–603. https://doi.org/10.1109/MASS62177.2024.00095
[31] H.M. Nguyen, E.W. Cooper, K. Kamei, Borderline over-sampling for imbalanced data classification, Int. J. Knowl. Eng. Soft Data Paradig. 3 (1) (2011) 4–21.
[32] K. Yu, K. Nguyen, Y. Park, Flexible and robust real-time intrusion detection systems to network dynamics, IEEE Access 10 (2022) 98959–98969. https://doi.org/10.1109/ACCESS.2022.3199375
[33] T. Kim, W. Pak, Real-time network intrusion detection using deferred decision and hybrid classifier, Future Gener. Computer Syst. 132 (2022) 51–66.
[34] E. Murtaj, F. Marcantoni, M. Loreti, M. Quadrini, H.-F. Witschel, Real-time intrusion detection via machine learning approaches, in: Di Martino, S., Sansone, C., Masciari, E., Rossi, S., Gravina, M. (Eds.), Proceedings of the Ital-IA Intelligenza Artificiale Thematic Workshops co-located with the 4th CINI National Lab AIIS Conference on Artificial Intelligence (Ital-IA 2024), Naples, Italy, 1192, CEUR Workshop Proceedings, 2024, pp. 200–205.
[35] A.H. Lashkari, Y. Zang, G. Owhuo, M. Mamun, G. Gil, CICFlowMeter, GitHub.[vid. 2021-08-10]. Dostupné z: https://github.com/ahlashkari/CICFlowMeter/blob/master/ReadMe.txt (2017).
[36] U.o. N.B. Canadian Institute for Cybersecurity, CSE-CIC-IDS2018 on AWS, 2018, Accessed: 2023-11-09, https://www.unb.ca/cic/datasets/ids-2018.html.
[37] L.N. Dayanand, B. Ghorbani, S. Vaghri, A survey on the use of GNS3 for virtualizing computer networks, International Journal of Computer Science and Engineering 5 (1) (2016) 49–58.
[38] K. Ben Kalboussi, F. Barika Ktata, I. Amous, A simulation framework for IoT networks intrusion and penetration testing, in: International Conference on Digital Technologies and Applications, Springer, 2023, pp. 252–263.
[39] V. Rajaravivarma, Virtual local area network technology and applications, in: Proceedings the Twenty-Ninth Southeastern Symposium on System Theory, IEEE, 1997, pp. 49–52.
[40] T.A. Tang, L. Mhamdi, D. McLernon, S.A.R. Zaidi, M. Ghogho, Deep learning approach for network intrusion detection in software defined networking, in: 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM), 2016, pp. 258–263. https://doi.org/10.1109/WINCOM.2016.7777224
[41] R. Bala, R. Nagpal, A review on kdd cup99 and nsl nsl-kdd dataset, Int. J. Adv. Res. Computer Sci. 10 (2) (2019).
[42] B. Isyaku, M.S. Mohd Zahid, M. Bte Kamat, K. Abu Bakar, F.A. Ghaleb, Software defined networking flow table management of openflow switches performance and security challenges: a survey, Future Internet 12 (9) (2020) 147.
[43] H. Jadidbonab, A. Tomlinson, H.N. Nguyen, T. Doan, S. Shaikh, A realtime in-vehicle network testbed for machine learning-based ids training and validation, in: Workshop on AI and Cybersecurity: co-located with 41st SGAI International Conference on Artificial Intelligence (SGAI 2021), CEUR Workshop Proceedings, 2022.
[44] D.V. Rishvanth, K. Ganesan, et al., Design of an in-vehicle network (Using LIN, CAN and FlexRay), gateway and its diagnostics using vector CANoe, American Journal of Signal Processing 1 (2) (2011) 40–45.
[45] Ó. Pérez-Gil, R. Barea, E. López-Guillén, L.M. Bergasa, C. Gómez-Huélamo, R. Gutiérrez, A. Díaz-Díaz, Deep reinforcement learning based control for autonomous vehicles in CARLA, Multimed. Tools Appl. 81 (3) (2022) 3553–3576.
[46] S.T. Mehedi, A. Anwar, Z. Rahman, K. Ahmed, R. Islam, Dependable intrusion detection system for IoT: a deep transfer learning based approach, IEEE Trans. Ind. Inf. 19 (1) (2022) 1006–1017.
[47] F. Ullah, S. Ullah, G. Srivastava, J.C.-W. Lin, IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic, Digital Commun. Netw. 10 (1) (2024) 190–204.
[48] P. Wu, H. Guo, R. Buckland, A transfer learning approach for network intrusion detection, in: 2019 IEEE 4th International Conference on Big Data Analytics (ICBDA), IEEE, 2019, pp. 281–285.
[49] S. Latif, W. Boulila, A. Koubaa, Z. Zou, J. Ahmad, DTL-IDS: an optimized intrusion detection framework using deep transfer learning and genetic algorithm, J. Netw. Computer Appl. 221 (2024) 103784.
[50] W. Li, Using genetic algorithm for network intrusion detection, Proceedings of the United States department of energy cyber security group 1 (1) (2004) 8.
[51] M.A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: a new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning, IEEE Access 10 (2022) 40281–40306.
[52] H. Kim, S. Park, H. Hong, J. Park, S. Kim, A transferable deep learning framework for improving the accuracy of internet of things intrusion detection, Future Internet 16 (3) (2024) 80.
[53] J.M. Peterson, J.L. Leevy, T.M. Khoshgoftaar, A review and analysis of the bot-iot dataset, in: 2021 IEEE International Conference on Service-Oriented System Engineering (SOSE), IEEE, 2021, pp. 20–27.
[54] T.-N. Hoang, D. Kim, Supervised contrastive ResNet and transfer learning for the in-vehicle intrusion detection system, Expert Syst. Appl. 238 (2024) 122181.
[55] U.K. Lilhore, P. Manoharan, S. Simaiya, R. Alroobaea, M. Alsafyani, A.M. Baqasah, S. Dalal, A. Sharma, K. Raahemifar, HIDM: Hybrid intrusion detection model for industry 4.0 networks using an optimized CNN-LSTM with transfer learning, Sensors 23 (18) (2023) 7856.

[56] G. Guo, X. Pan, H. Liu, F. Li, L. Pei, K. Hu, An IoT intrusion detection system based on TON IoT network dataset, in: 2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC), IEEE, 2023, pp. 0333–0338.
[57] G. D'Angelo, F. Palmieri, Network traffic classification using deep convolutional recurrent autoencoder neural networks for spatial–temporal features extraction, J. Netw. Computer Appl. 173 (2021) 102890.
[58] F. Zare, P. Mahmoudi-Nasr, Feature engineering methods in intrusion detection system: a performance evaluation, Int. J. Eng. Trans. A 36 (7) (2023) 1343–1353.
[59] A. Thakkar, R. Lohiya, A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions, Artif. Intell. Rev. 55 (1) (2022) 453–563.
[60] H. Liu, M. Zhou, X.S. Lu, C. Yao, Weighted gini index feature selection method for imbalanced data, in: 2018 IEEE 15th International Conference on Networking, Sensing and Control (ICNSC), IEEE, 2018, pp. 1–6.
[61] A. Thakkar, N. Kikani, R. Geddam, Fusion of linear and non-linear dimensionality reduction techniques for feature reduction in LSTM-based intrusion detection system, Appl. Soft Comput. 154, (2024) 111378.
[62] A. Yulianto, P. Sukarno, N.A. Suwastika, Improving adaboost-based intrusion detection system (IDS) performance on CIC IDS 2017 dataset, in: J. Phys., 1192, IOP Publishing, 2019, p. 012018.
[63] M. Almseidin, J. Al-Sawwa, M. Alkasassbeh, Generating a benchmark cyber multi-step attacks dataset for intrusion detection, J. Intell. Fuzzy Syst. 43 (3) (2022) 3679–3694.
[64] A. Davis, S. Gill, R. Wong, S. Tayeb, Feature selection for deep neural networks in cyber security applications, in: 2020 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS), IEEE, 2020, pp. 1–7.
[65] D.-W. Kim, G.-Y. Sin, K. Kim, J. Kang, S.-Y. Im, M.-M. Han, Network traffic synthesis and simulation framework for cybersecurity exercise systems, Computers Mater. Continua 80 (3) (2024).
[66] S. Garcia, A. Parmisano, M.J. Erquiaga, IoT-23: A labeled dataset with malicious and benign IoT network traffic (Version 1.0.0), 2020, Zenodo, Dataset, https://doi.org/10.5281/zenodo.4743746
[67] I. Sharafaldin, A.H. Lashkari, S. Hakak, A.A. Ghorbani, Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: 2019 International Carnahan Conference on Security Technology (ICCST), 2019, pp. 1–8. https://doi.org/10.1109/CCST.2019.8888419
[68] K. Sharanagowda, A study on the client server architecture and its usability, IOSR J. Computer Eng. (IOSR-JCE) 24 (4, Ser. I) (2022) 73–76. Assistant Professor, Department of Computer Science, Government First Grade College for Women, Raichur :pin 584101, http://www.iosrjournals.org.
[69] M. Pilgrim, Serializing python objects, in: Dive into Python 3, Springer, 2009, pp. 205–223.
[70] J. Hunt, J. Hunt, Sockets in python, Adv. Guide Python 3 Programm. (2019) 457–470.
[71] Y. Yu, A. Calagna, P. Giaccone, C.F. Chiasserini, Tcp connection management for stateful container migration at the network edge, in: 2023 21st Mediterranean Communication and Computer Networking Conference (MedComNet), IEEE, 2023, pp. 151–157.
[72] A.I. Toolify, Efficient Transfer of Python Objects via Sockets, 2023, (https://www.toolify.ai/gpts/efficient-transfer-of-python-objects-via-sockets-133218). Accessed: 2024-11-09.
[73] C. Grellmann, S. Bitzer, J. Neumann, L.T. Westlye, O.A. Andreassen, A. Villringer, A. Horstmann, Comparison of variants of canonical correlation analysis and partial least squares for combined analysis of MRI and genetic data, Neuroimage 107 (2015) 289–310.
[74] GeeksforGeeks, Partial least squares (pls) canonical, (https://www.geeksforgeeks.org/partial-least-squares-pls-canonical/). Accessed: 12/04/2024.
[75] R. Chapaneri, S. Shah, Detection of malicious network traffic using convolutional neural networks, in: 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT), IEEE, 2019, pp. 1–6.
[76] S.N. Pakanzad, H. Monkaresi, Providing a hybrid approach for detecting malicious traffic on the computer networks using convolutional neural networks, in: 2020 28th Iranian Conference on Electrical Engineering (ICEE), IEEE, 2020, pp. 1–6.
[77] L. Mohammadpour, T.C. Ling, C.S. Liew, A. Aryanfar, A survey of CNN-based network intrusion detection, Appl. Sci. 12 (16) (2022) 8162.
[78] O.S. Younes, A hybrid deep learning model for detecting DDoS flooding attacks in SIP-based systems, Comput. Netw. 240 (2024) 110146.
[79] P. Baldi, P.J. Sadowski, Understanding dropout, Adv. Neural Inf. Process. Syst. 26 (2013).
[80] S. Reddy, K.T. Reddy, V. ValliKumari, Optimization of deep learning using various optimizers, loss functions and dropout, Int. J. Recent Technol. Eng. 7 (2018) 448–455.
[81] A. Parmar, R. Katariya, V. Patel, A review on random forest: an ensemble classifier, in: International Conference on Intelligent Data Communication Technologies and Internet of Things (ICICI) 2018, Springer, 2019, pp. 758–763.
[82] M.P. LaValley, Logistic regression, Circulation 117 (18) (2008) 2395–2399.
[83] G. Guo, H. Wang, D. Bell, Y. Bi, K. Greer, KNN model-based approach in classification, in: On the Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE: OTM Confederated International Conferences, CoopIS, DOA, and ODBASE 2003, Catania, Sicily, Italy, November 3–7, 2003. Proceedings, Springer, 2003, pp. 986–996.
[84] S. Naiem, A.E. Khedr, A.M. Idrees, M.I. Marie, Enhancing the efficiency of gaussian naïve bayes machine learning classifier in the detection of ddos in cloud computing, IEEE Access 11 (2023) 124597–124608.
[85] L. Yang, A. Shami, On hyperparameter optimization of machine learning algorithms: theory and practice, Neurocomputing 415 (2020) 295–316.
[86] X.U. Manfei, D. Fralick, J.Z. Zheng, B. Wang, F. Changyong, et al., The differences and similarities between two-sample t-test and paired t-test, Shanghai Arch. Psychiatry 29 (3) (2017) 184.
[87] T.K. Kim, T test as a parametric statistic, Korean J. Anesthesiol. 68 (6) (2015) 540–546.
[88] F. Rustam, I. Obaidat, A.D. Jurcut, MULTI-LF: A Unified Continuous Learning Framework for Real-Time DDoS Detection in Multi-Environment Networks, 2025, arXiv:2504.11575
[89] I. Sharafaldin, A. Habibi Lashkari, A.A. Ghorbani, A detailed analysis of the cicids2017 data set, in: Information Systems Security and Privacy: 4th International Conference, ICISSP 2018, Funchal-Madeira, Portugal, January 22–24, 2018, Revised Selected Papers 4, Springer, 2019, pp. 172–188.
[90] B. Babayigit, M. Abubaker, Towards a generalized hybrid deep learning model with optimized hyperparameters for malicious traffic detection in the industrial internet of things, Eng. Appl. Artif. Intell. 128 (2024) 107515.
[91] F.A. Demmese, S. Shajarian, S. Khorsandroo, Transfer learning with ResNet50 for malicious domains classification using image visualization, Disc. Artif. Intell. 4 (1) (2024) 52.
[92] Z. Wu, H. Zhang, P. Wang, Z. Sun, RTIDS: a robust transformer-based approach for intrusion detection system, IEEE Access 10 (2022) 64375–64387. https://doi.org/10.1109/ACCESS.2022.3182333
[93] S. Ullah, J. Ahmad, M.A. Khan, M.S. Alshehri, W. Boulila, A. Koubaa, S.U. Jan, M.M.I. Ch, TNN-IDS: transformer neural network-based intrusion detection system for MQTT-enabled IoT networks, Comput. Netw. 237 (2023) 110072.
[94] A. Triantafyllou, P. Sarigiannidis, T.D. Lagkas, Network protocols, schemes, and mechanisms for internet of things (iot): features, open challenges, and trends, Wireless Commun. Mobile Comput. 2018 (1) (2018) 5349894.
[95] A. Apon, M. Baker, Network technologies, Int. J. High Perform. Comput. Appl. 15 (2) (2001) 102–114.